Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3656176

PDF Download
3656176.pdf
10 February 2026
Total Citations: 6
Total Downloads: 1186

Published: 10 May 2024
Online AM: 08 April 2024
Accepted: 28 March 2024
Revised: 29 December 2023
Received: 16 June 2023

Citation in BibTeX format

RESEARCH-ARTICLE

HierCGRA: A Novel Framework for Large-scale CGRA with Hierarchical Modeling and Automated Design Space Exploration

SICHAO CHEN, Fudan University, Shanghai, China

CHANG CAI, Fudan University, Shanghai, China

SU ZHENG, Chinese University of Hong Kong, Hong Kong, Hong Kong

JIANGNAN LI, Fudan University, Shanghai, China

GUOWEI ZHU, Fudan University, Shanghai, China

JINGYUAN LI, Fudan University, Shanghai, China

View all

Open Access Support provided by:

Fudan University

Chinese University of Hong Kong

ACM Transactions on Reconfigurable Technology and Systems, Volume 17, Issue 2 (June 2024)
https://doi.org/10.1145/3656176
EISSN: 1936-7414

HierCGRA: A Novel Framework for Large-scale CGRA with Hierarchical Modeling and Automated Design Space Exploration

SICHAO CHEN, School of Microelectronics, Fudan University, Shanghai, China
CHANG CAI, School of Microelectronics, Fudan University, Shanghai, China
SU ZHENG, Department of Computer Science and Engineering, The Chinese University of Hong Kong, Hong Kong, Hong Kong
JIANGNAN LI, School of Microelectronics, Fudan University, Shanghai, China
GUOWEI ZHU, School of Microelectronics, Fudan University, Shanghai, China
JINGYUAN LI, School of Microelectronics, Fudan University, Shanghai, China
YAZHOU YAN, School of Microelectronics, Fudan University, Shanghai, China
YUAN DAI, School of Microelectronics, Fudan University, Shanghai, China
WENBO YIN, School of Microelectronics, Fudan University, Shanghai, China
LINGLI WANG, School of Microelectronics, Fudan University, Shanghai, China

Coarse-grained reconfigurable arrays (CGRAs) are promising design choices in computation-intensive domains, since they can strike a balance between energy efficiency and flexibility. A typical CGRA comprises processing elements (PEs) that can execute operations in applications and interconnections between them. Nevertheless, most CGRAs suffer from the ineffectiveness of supporting flexible architecture design and solving large-scale mapping problems. To address these challenges, we introduce HierCGRA, a novel framework that integrates hierarchical CGRA modeling, Chisel-based Verilog generation, LLVM-based data flow graph (DFG) generation, DFG mapping, and design space exploration (DSE). With the graph homomorphism (GH) mapping algorithm, HierCGRA achieves a faster mapping speed and higher PE utilization rate compared with the existing state-of-the-art CGRA frameworks. The proposed hierarchical mapping strategy achieves 41× speedup on average compared with the ILP mapping algorithm in CGRA-ME. Furthermore, the automated DSE based on Bayesian optimization achieves a significant performance improvement by the heterogeneity of PEs and interconnections. With these features, HierCGRA enables the agile development for large-scale CGRA and accelerates the process of finding a better CGRA architecture.

CCS Concepts: • **Hardware → Reconfigurable logic and FPGAs**; **Software tools for EDA**;

Additional Key Words and Phrases: CGRA modeling, hierarchical mapping, automated design space exploration

S. Chen and C. Cai contributed equally to this research.

Authors' addresses: S. Chen, C. Cai, J. Li, G. Zhu, J. Li, Y. Yan, Y. Dai, W. Yin, and L. Wang (Corresponding author), School of Microelectronics, Fudan University, No. 825 Zhangheng Road, Shanghai, China, 201203; e-mails: 20112020050@fudan.edu.cn, 21212020050@m.fudan.edu.cn, jnli22@m.fudan.edu.cn, gwzhu20@fudan.edu.cn, jingyuanli22@m.fudan.edu.cn, yzyan21@m.fudan.edu.cn, daiy21@m.fudan.edu.cn, wbyin@fudan.edu.cn, llwang@fudan.edu.cn; S. Zheng, Department of Computer Science and Engineering, The Chinese University of Hong Kong, Shatin, NT, Hong Kong, China, 999077; e-mail: szheng22@cse.cuhk.edu.hk.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 1936-7406/2024/05-ART35
https://doi.org/10.1145/3656176

ACM Reference Format:
Sichao Chen, Chang Cai, Su Zheng, Jiangnan Li, Guowei Zhu, Jingyuan Li, Yazhou Yan, Yuan Dai, Wenbo Yin, and Lingli Wang. 2024. HierCGRA: A Novel Framework for Large-scale CGRA with Hierarchical Modeling and Automated Design Space Exploration. *ACM Trans. Reconfig. Technol. Syst.* 17, 2, Article 35 (May 2024), 31 pages. https://doi.org/10.1145/3656176

1 INTRODUCTION

With the slowing down of Moore's Law and Dennard scaling, the demand for energy-efficient architectures becomes urgent [20, 29]. **Application-Specific Integrated Circuits (ASICs)** provide high performance and energy efficiency for specific applications. However, the limited configurability of ASICs reduces their flexibility and therefore makes them less applicable in fast-evolving fields, especially for machine learning [30, 53, 56], where novel algorithmic approaches and models evolve aggressively. **Field Programmable Gate Arrays (FPGAs)** are a popular alternative for these fast-changing applications. Nevertheless, FPGAs suffer from high power and delay overheads due to their bit-level reconfigurability. **Coarse-Grained Reconfigurable Architectures (CGRAs)** can strike a balance between energy efficiency and flexibility [1, 39], making them ideal for domain-specific applications. Moreover, CGRAs can achieve higher energy efficiency than FPGAs in the target computation-intensive domains due to their word-level and per-cycle reconfigurability [1, 39]. These advantages make CGRAs garner much attention in emerging fields, such as machine learning [23, 32, 62], cryptographic algorithm acceleration [18, 47, 48], and data processing [11, 19].

Designers can make tradeoffs between flexibility and efficiency to fit applications in different domains. A typical CGRA architecture consists of an array of **process elements (PEs)** and interconnects between them, scaled up by increasing the length and width of the array [2, 51, 52]. CGRAs with high flexibility, such as ADRES [42] and HyCUBE [31], have limited scales and extensibility. The complicated interconnections raise the difficulty of scaling up, and the diverse data flows increase the complexity of the mapping procedure. Existing large-scale CGRAs such as Cryptoraptor [48] and DT-CGRA [21] usually have limited reconfigurability due to the specialization of PEs and interconnections, decreasing the generality of these CGRAs. AHA [36] is an extensible island-style CGRA with sufficient reconfigurability. However, the interconnections based on routing channels and switch boxes cause latency overheads and redundancy like the interconnections in FPGAs. **General Interconnection Blocks (GIBs)** [50] can provide scalable and reconfigurable interconnections with low redundancy and latency in reconfigurable hardware, which are applicable in different-scale CGRA designs. GIBs are also used in other CGRA frameworks, such as TRAM [46].

The mapper, which is responsible for mapping algorithms onto CGRAs, plays an influential role in boosting the performance and efficiency of the hardware. There are various algorithms for CGRA application **placement and routing (PnR)**, most of which are based on **simulated annealing (SA)** algorithms [2, 22], **integer linear programming (ILP)** [24, 55], **satisfiability modulo theory (SMT)** [17], and graph algorithms [3, 9, 12, 27, 28, 40, 58]. The graph algorithms define the problem as mapping the DFG into the **routing resource graph (RRG)**, where the DFG represents the application, and the RRG represents the CGRA hardware.

Existing CGRA frameworks suffer from the ineffectiveness of supporting flexible architecture design and solving large-scale mapping problems. To address the above disadvantages, we propose HierCGRA, a flexible and scalable CGRA framework that allows complex interconnection generation and hierarchical CGRA modeling. This work is an upgraded version of Reference [60]. Our contributions include:

Table 1. Comparison of Different CGRA Frameworks

Framework	Kernel code [34]	Assumed execution model [34]		Freedom of architecture [34]	Hierarchical mapping	Automated DSE
		Spatially configured	Time multiplexed			
CGRA-ME [2]	C	✓	✓	Any	×	×
OpenCGRA [51]	C/C++ Python DSL	×	✓	Any	×	✓
HierCGRA	C/C++ Python	✓	✓	Any	✓	✓

— We propose HierCGRA, a novel framework that integrates hierarchical CGRA modeling, Chisel-based Verilog generation, LLVM-based DFG generation, DFG mapping, architecture evaluation, and DSE. Table 1 compares the features of this work and other CGRA frameworks [34].
— The compiling flow in HierCGRA enables the generation of DFG either by extracting compute kernels from C programs based on LLVM or modeling by the proposed Python-based data flow description.
— The hierarchical mapping algorithm based on graph homomorphism achieves a faster mapping speed with higher PE utilization on existing benchmarks and shows promising scalability for large-scale DFG and heterogeneous CGRA architecture. The novel connectivity-aware packing and genetic partition algorithm effectively reduce the searching space, accelerating the mapping flow.
— We propose an automated two-phase DSE algorithm based on Bayesian optimization to explore CGRA design parameters that can achieve multi-objective optimization, enabling designers to find a better design systematically. Experimental results show that our Bayesian-optimization-based DSE achieves a high PE utilization rate (up to 70%) and significant average II-area product reduction (14.16%–46.39% in different CGRA scales) by the heterogeneity of architecture and interconnects.

The rest of this article is organized as follows: In Section 2, we provide a high-level overview of the HierCGRA framework and briefly describe the function of each tool. The CGRA modeling process is proposed in Section 3. The compiling flow integrated with the hierarchical mapping algorithm is described in Section 4. Section 5 introduces the architecture evaluation and DSE part of HierCGRA. Section 6 presents the experimental results compared with existing CGRA frameworks. Section 7 concludes this article with future work.

2 OVERVIEW OF HIERCGRA FRAMEWORK

The overall HierCGRA framework is shown in Figure 1. HierCGRA is a powerful toolset for agile CGRA development, consisting of three parts: application compiling flow, architecture modeling, and DSE. Each solid box represents a tool, including modeling, DFG generator, mapper, DSE engine, Chisel-tester2 [54], and Verilog generator.

Application compiling flow supports two forms of inputs: (1) Data flow description based on Python or (2) C programs annotated with pragmas. The compute kernel is extracted from applications and then translated to a **data-flow graph (DFG)** by DFG generator as input of mapper. We model the CGRA hardware with a Scala-based architecture description, which allows a hardware engineer with no programming experience to design a large-scale, heterogeneous CGRA architecture easily. This architecture description supports the modeling of highly flexible and scalable CGRA architectures, rather than architectures consisting of just a matrix of PEs and node-to-node interconnects. Rich design parameters are integrated into our architecture templates to facilitate automated architecture generation and rapid DSE. The architecture generator estimates the

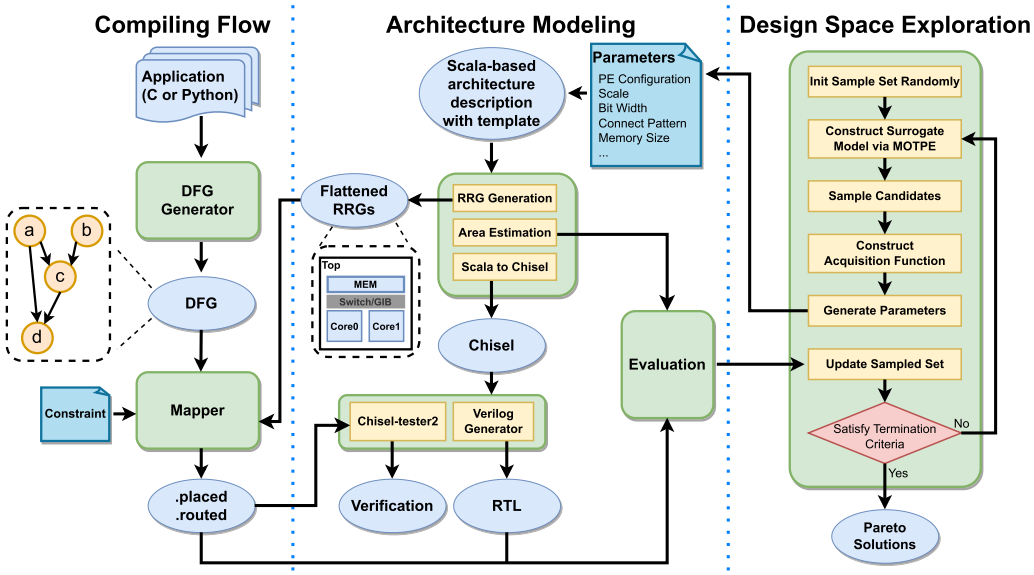

Fig. 1. Overview of HierCGRA framework.

hardware area and exports several flattened RRG files and a Chisel file to describe the hardware. The RRG files are utilized in the mapper as a description of the architecture. Besides the DFG and RRG files, the mapper also needs a constraint file, which allows the user to adjust the mapping process for custom CGRA architectures. The mapping result includes configuration information, **initial interval (II)**, and resource utilization rate. The Chisel file is fed into the Chisel-tester2 [54] and Verilog generator. Besides the Chisel file of the CGRA hardware, Chisel-tester2 [54] also needs the mapping result to verify the hardware function. The Verilog generator produces a Verilog file of the CGRA hardware for **Register-Transfer-Level (RTL)** synthesis. In the DSE process, we parameterize a CGRA architecture and adjust the architecture automatically by evaluating the results of modeling and mapping.

3 ARCHITECTURE MODELING

Figure 2(a) shows a CGRA architecture with multiple hierarchies, which can be classified into two types: units and modules. Units, such as **function units (FUs)**, multiplexers, and registers, are at the bottom hierarchy of the CGRA architecture. Modules are at higher hierarchies of the architecture—they can contain units or lower-hierarchy modules. In Figure 2(a), the *Top* module is at the highest hierarchy. *Core0* and *Core1* modules are sub-modules of the *Top* module, and each of them contains several *Block* modules and FUs. *Block* modules also contain multiple FUs. All units and modules contain two essential parts: ports and connections. Ports define their inputs and outputs, while connections link different hardware components. Figure 2(c) presents the internal construction of an FU, which contains an arithmetic operator and multiple ports, as well as automatically generated inner connections and config-bits.

In a CGRA architecture, some modules only consist of multiplexer units. In these modules, data can be routed from one port to another by configuration bits, which alleviates the interconnection complexity between modules and units, so they are called as interconnection modules. They can also be classified into two types: switches and GIBs [50]. A switch (Figure 2(b)) can connect other switches by routing channels in only one direction, while a GIB (Figure 3) has interconnections between other GIBs in four directions. In a GIB, the *Pins* on its corners

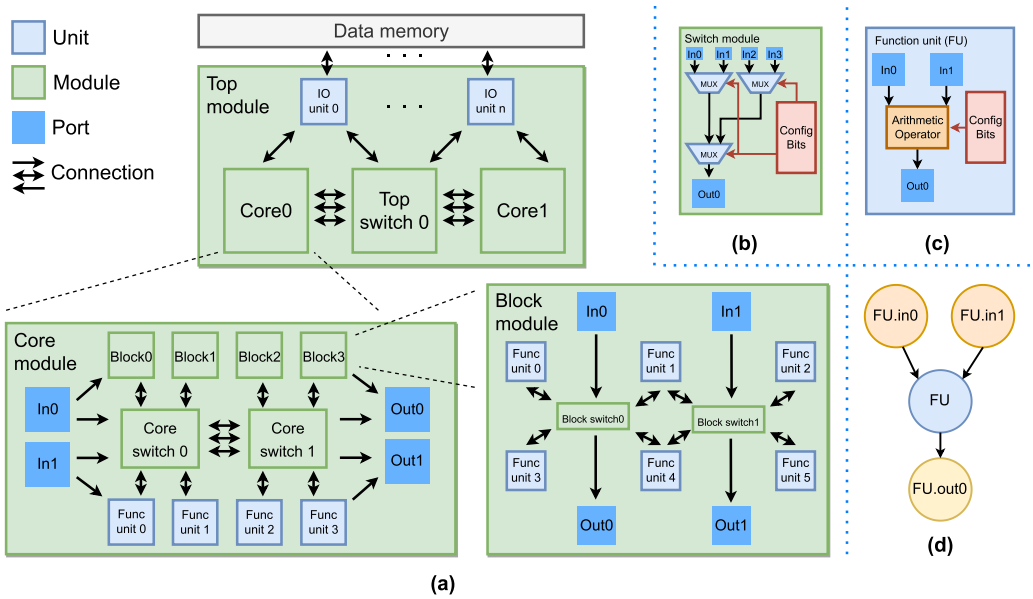

Fig. 2. An example of hierarchical CGRA architecture. (a) The hierarchical CGRA hardware. (b) Switch module. (c) Function unit (FU). (d) RRG of the function unit.

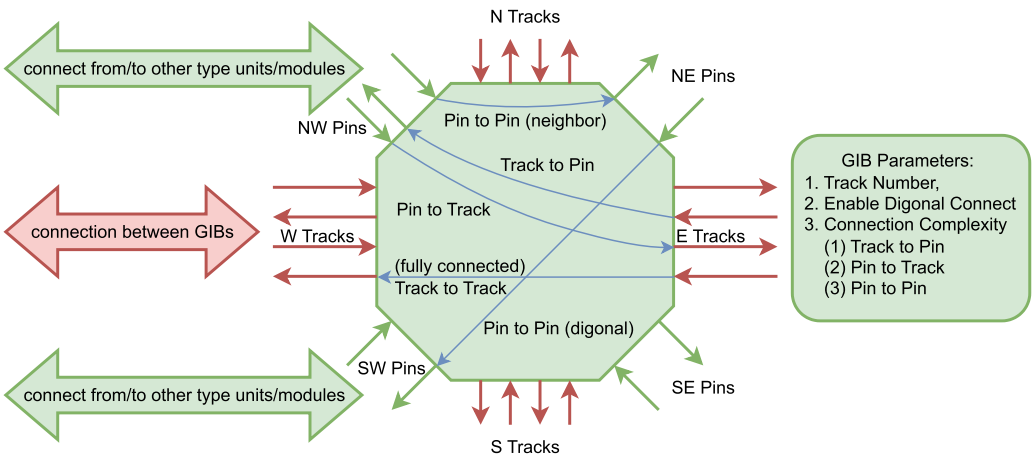

Fig. 3. GIB inner connections [50].

connect other non-interconnection modules, while the *Tracks* on its sides connect other GIBs. All interconnection modules can be described by the connection table of their input and output ports, which are parameterized in GIBs. So, when a developer uses GIBs as interconnection modules, greater interconnection flexibility can be easily achieved by tweaking these parameters without rebuilding the architecture.

When developers use the Scala architecture description, the attributes of modules and units, such as their IO ports, inner connection patterns, and arithmetic operations, are described and parameterized in Chisel as Scala functions. With this architecture modeling method, developers can construct a higher hierarchy module based on units and low-hierarchy modules to build a hierarchical CGRA architecture.

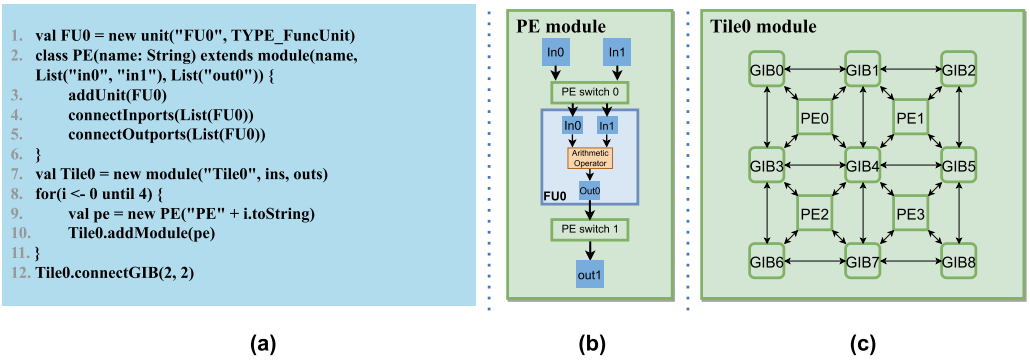

Fig. 4. CGRA modeling example. (a) Scala description example. (b) The PE module. (c) The 2 × 2 CGRA with GIB connections.

Figure 4 demonstrates a 2×2 CGRA with GIB interconnections constructed by Scala. Figure 4(a) shows some basic Scala architecture description syntax. In line 1, we define a FU named FU0, which is provided in our library. Then, in line 2 to line 6, we construct a PE module with 2 input ports, 1 output port, and a FU0. The *connectInports*() and *connectOutports*() functions in lines 4 and 5 tell the constructor to automatically generate switches for connecting the module's IO ports and units. Lines 7–12 construct the Tile0 module analogously. In line 7 to line 11, the previously built PE is utilized as a sub-module in our building process. In line 12, a function $connectGIB(row, col)$ is called for auto-generation of GIBs and connections according to PE's *row* and *col* number instead of declaring each GIB separately. The generated architecture is shown in Figure 4(b). Other interconnection architectures can also be generated by corresponding pre-built functions. For example, N2N interconnections can be implemented by calling function $connect(a, b)$, in which a and b are nodes in a module. The hardware model can be converted into two formats: Chisel file and RRG file.

In the Chisel file generation procedure, each hierarchy in a CGRA architecture is converted to a corresponding *Chisel module*. Then, their RTL implementations are produced by Chisel's Verilog generator. The switches, GIBs, connections, and ports are implemented as multiplexers, wires, and input/output ports in RTL. Meanwhile, the configuration bits and hardware control signals are automatically generated.

We use the graph data structure to describe RRG, which can be interpreted by the mapper. In the RRG generation process, all modules in the CGRA architecture are flattened to the unit level. All units are converted to nodes, and their connections are converted to edges. For example, an FU in Figure 2(c) consists of a unit-body node $v(FU)$, several unit-port nodes $v(FU.in0)$, $v(FU.in1)$, and $v(FU.out0)$ and edges $\vec{e}(FU.in0, FU)$, $\vec{e}(FU.in1, FU)$, $\vec{e}(FU, FU.out0)$, which is shown in Figure 2(d). Besides the top hierarchy module's RRG file, each sub-module in the top hierarchy module is converted to a separated RRG file to support the partition process detailed in Section 4.5.

4 APPLICATION COMPILING FLOW

4.1 DFG Generation

Two alternatives in HierCGRA can be used for DFG generation:

- Extracting the DFG for compute kernel from C application code via an LLVM-based compiler pass [37], similar to CGRA-ME [2] and CCF [15].
- Modelling the DFG using a Python-based data flow description.

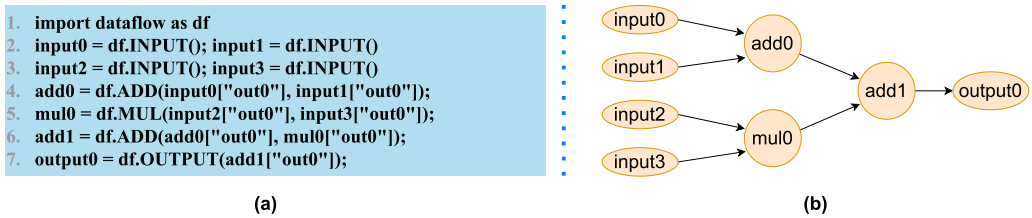

Fig. 5. Python-described DFG. (a) Python description. (b) Corresponding DFG.

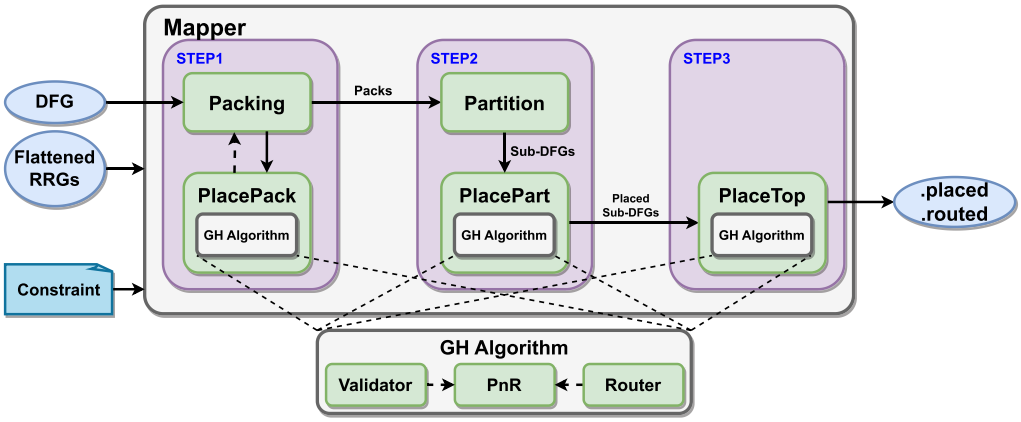

Fig. 6. Overview of DFG mapper.

In the LLVM-based compiler, we first extract the **intermediate representation (IR)** of a program, where each line represents an operation, such as *add* and *store*. Like CGRA-ME, we mark the C language applications with pragmas to indicate the compute kernel designated for CGRA and construct the DFG by presenting each operator with a node and connecting the operands to corresponding operators with edges. With this utility, we can easily convert existing C programs to DFGs that can be mapped to CGRAs.

The LLVM-based modeling method enables users to perform rapid modeling of the applications, but it has the disadvantage that the basic operations are limited to logical and arithmetic operations. Whereas the Python-based data flow description method supports the user to define operators that are not supported by C. Thus, it can support more CGRA designs and data flows than C-based representations. Figure 5 presents an example of the construction of a data flow. The internal representation of Python-based data flow description is a **function graph (FG)**, which consists of the basic operations and their operands. The FG is converted into a DFG by the pattern-matching process. Subsequently, after the mapping process, the nodes in DFG are mapped to the functions or their ports, and the edges in DFG are mapped to the connections.

4.2 Overview of DFG Mapper

We depict the mapping flow in Figure 6. Each rounded rectangle in the diagram represents a sub-tool of the mapper, the solid lines represent the direction of the data flow, and the dashed lines represent the calling relationship between them. There are three fundamental sub-tools in HierCGRA mapper: Packing, Partition, and PnR. We narrate the function of each sub-tool with the architecture in Figure 2 as an example. The Packing sub-tool, detailed in Section 4.4, aggregates DFG nodes into packs. The nodes obtained after the Packing are basic nodes that are compatible with a unit and packs that are compatible with a low-level module. A unit is at the bottom hierarchy of the

Table 2. Comparison of Different Hierarchical DFG Mappers

Mapper	PANORAMA [57]	CasMap [40]	HierCGRA Mapper
DFG Division Method	Spectral Clustering	Dijkstra-based Clustering	Connectivity-aware Packing Spectral Clustering,[1] GA-partition
RRG Contraction	–	✓	✓
N-subgraph-pattern Validation	–	–	✓
Mapping Algorithm	ILP[2]	ILP	Graph Homomorphism

[1]Only used to generate initial populations for GA-partition.
[2]Only used for high-level cluster mapping, the low-level mapping algorithm depends on the low-level mapper used by PANORAMA.

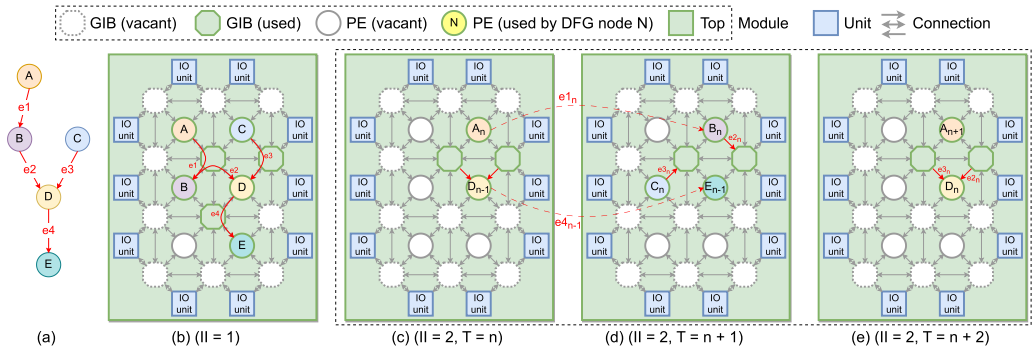

Fig. 7. Spatial mapping and temporal mapping. (a) The DFG to be mapped to CGRA. (b) The spatial mapping result (II = 1). (c) The temporal mapping result when II = 2 and T = n. (d) The temporal mapping result when II = 2 and T = n + 1. (e) The temporal mapping result when II = 2 and T = n + 2.

CGRA. A low-level module does not consist of other modules, so it only consists of units. Units and modules are introduced in Section 3. Then, the Partition sub-tool divides the above nodes into sub-graphs, each of which can be mapped into a sub-module in the top hierarchy, which is described in detail in Section 4.5. The Router and Validator are integrated into the PnR sub-tool, which is called by three programs in turn: PlacePack, PlacePart, and PlaceTop. The PlacePack program can map the packs obtained by the Packing sub-tool. The PlacePart program maps the sub-graphs obtained by the Partition sub-tool. The PlaceTop program maps all the remaining units, respectively. Each of the above three programs' running process depends on the results obtained from the previous one. Notably, The Packing sub-tool calls the PlacePack in the clustering phase to ensure that the aggregated packs can be mapped successfully. The details of the PnR sub-tool and how it supports the constraint file are stated in Section 4.6. Table 2 compares the features of our mapper and other two hierarchical DFG mappers: PANORAMA [57] and CasMap [40]. Our DFG mapper supports spatial and temporal mapping. The spatial mapping process maps the DFG to RRG, while the temporal mapping process maps it to **Modulo RRG (MRRG)**. MRRGs are generated by mapper from RRG files. The II of spatial mapping is 1, while temporal mapping's II is larger than 1. Figure 7 shows the difference between these two mapping modes. In Figure 7(b), the II is 1, and all DFG edges are mapped to connections and GIBs. Figures 7(c)–(e) show the mapping result when II = 2. Two pairs of adjacent DFG nodes are mapped to the same PE in different cycles: $A - B$ and $D - E$, so they can directly transfer data using the registers in their PEs. In the first cycle (shown in Figures 7(c) and (e)), the DFG nodes A and D store the data in the registers, and in the second cycle (shown in Figure 7(d)), the DFG nodes B and E read the data in these registers. Thus, the DFG edge $e1$ and $e4$ are implemented by registers, while the $e2$ and $e3$ are still mapped to interconnection resources.

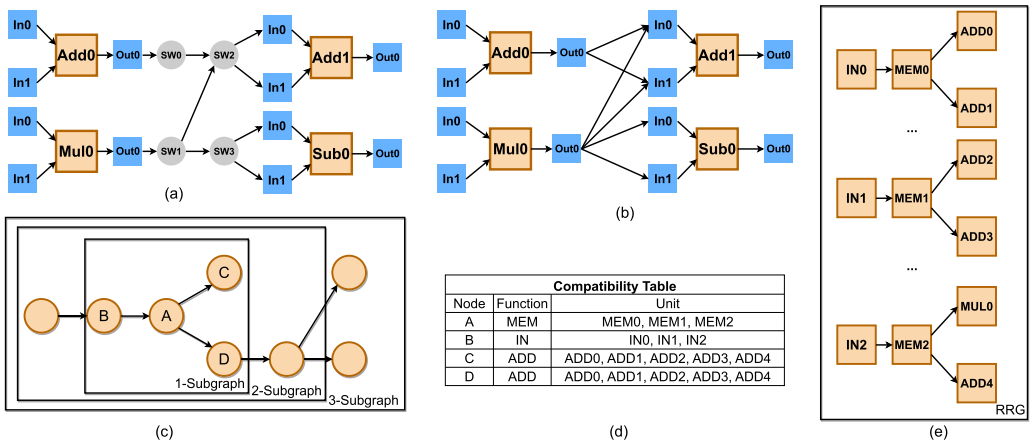

Fig. 8. Example of RRG contraction and *N-subgraph-pattern validation* (*NSPV*). (a) Original RRG. (b) Contracted RRG. (c) *N-subgraph* of a DFG. (d) Compatibility table of the DFG. (e) Several subgraphs in an RRG.

4.3 RRG Contraction and N-subgraph-pattern Validation

The mapping flow relies on the RRG (or MRRG in temporal mapping mode) contraction procedure. RRG or MRRG nodes can be classified into computing resources and interconnection resources. The computing resources include the FUs and their ports. The nodes in RRG/MRRG excluding computing resources are called interconnection resources. In the contraction procedure, the interconnection resources are replaced by the edges between the ports of the units. Each new edge connects a source output port of a unit and an input port of a unit that is reachable from the source port. Figure 8(b) presents an example of the contracted RRG obtained from Figure 8(a). Previous DFG mappers based on graph minor approaches, such as Reference [12], run the RRG contraction during placement and routing. In our mapper, the RRG contraction process is executed before them. The possible congestion in the routing procedure is ignored in the contracted RRG due to the removal of the multiplexers, and the original graph minor mapping problem is transformed into a graph homomorphism problem, so the mapping efficiency is improved. The contracted RRG can be used in the *N-subgraph-pattern validation* and placing phase for the rapid pruning of unfeasible solutions.

N-subgraph-pattern validation (*NSPV*) is an essential procedure to reduce invalid placement trials. The *N-subgraph* of a node v is defined as the subgraph consisting of v and all nodes whose distances to v are less than or equal to N. Figure 8(c) shows the *1-subgraph*, *2-subgraph*, and *3-subgraph* of node A in a DFG. To illustrate the *NSPV*, we first define the *N-subgraph-pattern compatibility* of two nodes. A node v_d in the DFG is *N-subgraph-pattern-compatible* with a node v_r in the contracted RRG (or MRRG) if the *N-subgraph* of v_d can be placed in the N-subgraph of v_r. In the *NSPV* procedure, we check whether each node in the DFG is *N-subgraph-pattern-compatible* with every node in the contracted RRG/MRRG. If a node v_d in the DFG is not *N-subgraph-pattern-compatible* with a node v_r in the contracted RRG/MRRG, then we erase v_r from the compatibility table of v_d. Figure 8 presents a specific example of the utilization of *NSPV* in the placement process. Figures 8(c), (d), and (e) show an example of a DFG, the configuration information of the nodes in DFG, and several parts of the sub-graph of the RRG. We omit the validation of the port node in RRG and let $N = 1$ simplify the problem description. When validating node A, MEM2 is pruned off because C and D in its *1-subgraph* cannot complete mapping simultaneously. If node C has been mapped to $ADD0$ in RRG in the previous mapping step, then $MEM1$ would also be

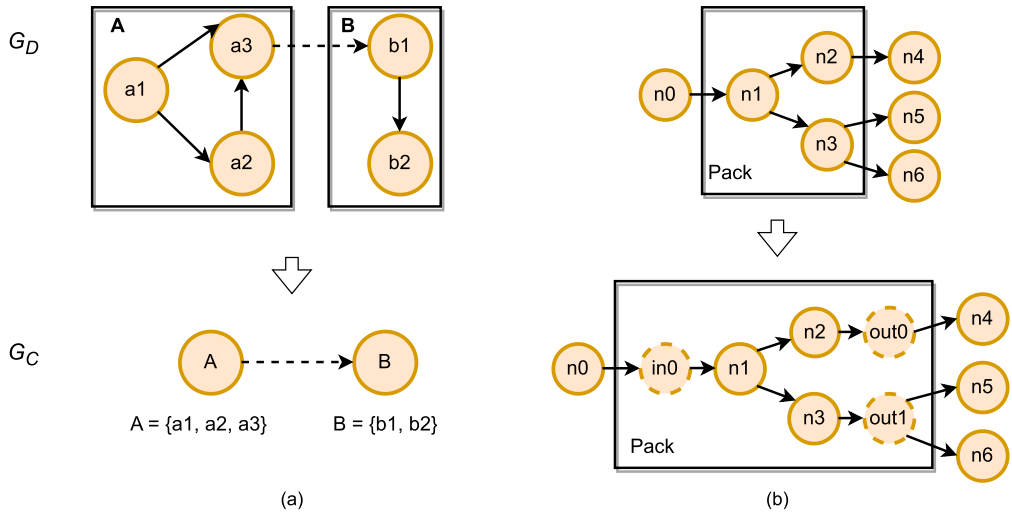

Fig. 9. Example of the coarsening procedure and *insertPort*() function. (a) The coarsening procedure from G_D to G_C. (b) Example of *insertPort*() function.

erased in the *NSPV* process. Both of the above functions are embedded in *Validator*. PANORAMA [57] is a high-level mapping framework that uses the DFG partition algorithm to guide low-level mapping, so the RRG contraction and *NSPV* support depend on the guided low-level mapper. CasMap [40] has a similar RRG contraction process but does not support the *NSPV*.

4.4 Connectivity-aware Packing

The connectivity-aware packing algorithm is used to group the DFG nodes into packs. The DFG can be denoted as $G_D = < V_D, E_D >$, where V_D is the node set and E_D is the edge set. We define pack P as a subset of V_D, and each node $v \in P$ can be placed into the same low-level module, which does not contain other modules, while for each edge $\vec{e}(v_1, v_2)$, $\vec{e} \in E_D, v_1 \in P, v_2 \in P$ can be routed within RRG (or MRRG). Our algorithm is inspired by the greedy search idea in the VPACK algorithm [43]. We extend this idea with a connectivity-aware coarsening process and a breadth-first search scheme. After this, a tabu search algorithm [5] based on **simulated annealing (SA)** [8] is executed to further optimize the number of cutting edges within the packs to reduce the time required for placement.

For CGRA packing, we design a coarsening procedure to pre-pack the DFG nodes and reduce the packing complexity, which consists of two steps:

- **Classifying E_D:** Classify the edges in E_D into *intra-pack* edges E_{intra} and *inter-pack* edges E_{inter}, where $\forall \vec{e}(v_1, v_2) \in E_{intra}$, $\vec{e}$ can only be routed in one low-level module that does not contain other modules. That is, v_1 and v_2 cannot be placed into different low-level modules. *Inter-pack* edges can be routed in a low-level module or between different low-level modules.
- **Coarsening G_D:** Generate the coarsened graph $G_C = < V_C, E_C >$, where each coarsened node in V_C represents a subgraph in G_D. All nodes in these subgraphs are connected by *intra-pack* edges. E_C is E_{inter}.

Figure 9(a) illustrates the coarsening procedure. The arrows in solid and dashed lines present the E_{intra} and E_{inter}, respectively. The DFG nodes a1, a2, and a3 are pre-packed as the coarsened node A, while b1 and b2 are coarsened as B. The coarsened nodes are always too small, for *intra-pack*

ALGORITHM 1: Connectivity-aware Packing

```
Input: coarsened DFG G_C, RRG (or MRRG) G_B
Output: packed result
queResult = Vector();
queStatus = Stack();
while there are node in G_C unpacked do
    seed = selectSeed(G_C, queResult);
    initPack = createPack(∅, seed);
    queExplore = PriorityQueue();
    extendable = true;
    while extendable do
        extendable = false;
        candidates = searchCandidates(initPack);
        while candidates is not empty & iteration times < ζ do
            foreach node in candidates do
                extendedPack = createPack(initPack, node);
                if visited(extendedPack) then
                    continue;
                insertPort(extendedPack);
                if placePack(extendedPack, G_B) then // G_B is the input RRG/MRRG.
                    queExplore.push(extendedPack);
                    extendable = true;
        initPack = queExplore.front();
    if queExplore is empty & queStatus is not empty then
        queResult.pop_back();
        queStatus.pop_front();
    else if queExplore is not empty then
        queResult.push_back(queExplore.front());
        queExplore.pop_front();
        queStatus.push(queExplore);
return queResult;
```

edges are rare. The packing algorithm is applied to G_C to extend the coarsened nodes to packs, each of which is a subgraph of the DFG. We depict our packing procedure in Algorithm 1. In the loop, we use the *selectSeed*() function to pick a node from G_C and initialize *initPack*. The *selectSeed*() function traverses all the unpacked nodes in G_C and records the number of edges between each of them and the packed nodes, then returns the least connected unpacked node. The *initPack*'s data structure is a collection of nodes, such as A = {a1, a2, a3} in Figure 9(a). The priority queue *queExplore* contains possible packing for *initPack*. It is prioritized according to the ratio cuts, the same as in spectral clustering [10]. The *searchCandidates*() function sorts the unpacked nodes according to the number of shared edges with the nodes in *initPack*. Nodes with more shared edges are prioritized for attempts, since this could reduce the ratio cuts. This function will additionally add a null node for subsequent individual validation of the *initPack*. We then add the *initPack* with nodes from *candidates* and get the *extendedPack*. The function *insertPort*(), illustrated in Figure 9(b), is used to insert port nodes for the subgraph in a pack. After that, the resulting subgraph is verified to be mappable to a module in the RRG (or MRRG) by *placePack*(). This ensures the pack's incoming and outcoming edges can be mapped to the RRG (or MRRG). The *placePack*() function tries the unused low-level modules to map the pack until success. Packs that pass the validation are pushed

into *queExplore*. Each time the *placePack*() function is run it records the nodes in the package, and then the *visited*() function prevents duplicate validation. Since each round of validation takes considerable time, one can limit the number of validations per round by customized constant ζ. After all *candidates* are traversed, we pick the front element of the priority queue *queExplore* as the *initPack* for the next round, until all validations in a given round have failed. If *queExplore* is not empty, then we add its front element to *queResult* and add the rest of the queue to *queStatus*. Otherwise, it means that the node could not be mapped individually, which is usually caused by that part of the optional node being occupied by other packs in earlier iteration. In this case, we backtrack using *queStatus* to release some of the nodes to try different combinations. This function returns the packing result *queResult*, which offers a preliminary result quickly, since it is essentially based on the greedy algorithm.

The result of connectivity-aware packing is considered the initialization of the SA process. Let $I = \{P_1, P_2, \ldots, P_k\}$ be the initialization of k-packs, P_{max} represents the pack with the most nodes. $V(P_i)$ is the subset of the nodes that connect with at least one node in P_i. There are three types of move operators in each iteration of the SA scheme:

- *Move 1:* Choose randomly $P_m \in \{P_1, P_2, \ldots, P_k\} - \{P_{max}\}$ and then move a node n_a from P_{max} to P_m, where $n_a \in V(P_{max})$.
- *Move 2:* Choose randomly $P_m, P_n \in \{P_1, P_2, \ldots, P_k\} - \{P_{max}\}, P_m \neq P_n$ and then swap node n_a and n_b, where $n_a \in V(P_n)$ and $n_b \in V(P_m)$.
- *Move 3:* Choose randomly $P_m, P_n \in \{P_1, P_2, \ldots, P_k\}, P_m \neq P_n$ and then swap node n_a and n_b, where $n_a \in V(P_n)$ and $n_b \in V(P_m)$. This operation only occurs when *Move1* and *Move2* fail to comply with the tabu list.

After the move operator, the *placePack*() function is called to ensure that the packs changed can still be mapped into a low-level module. This procedure is quite fast due to the small node number in packs and the high mapping success rate. Each time a node is moved from a subset P_m, a tabu list updates to forbid the node from being moved back to P_m for the next tabu tenure iterations. The tabu tenure is tuned adaptively according to the number of $V(P_m)$. The above measures are taken to ensure that the nodes are evenly distributed in the packs and to prevent invalid searching. Define E^P_{cut} as a subset of E_D, $\forall \vec{e}(v1, v2) \in E_D$, if $(v1 \in P \oplus v2 \notin P)$, $\vec{e} \in E_{cut}$, else $\vec{e} \notin E_{cut}$. Let C_i be the number of $E^{P_i}_{cut}$, $P_i \in \{P_1, P_2, \ldots, P_k\}$, and C_{max} is the maximum value among all C_i. The evaluation score in SA is that the resulting packs have the least C_{max}.

4.5 GA-partition Based on Pareto Improvement

The partition process can be applied to the nodes in the DFG or the sub-DFGs generated by previous packing or partition processes. This process can simplify the mapping problem when the DFG's scale is large. The outputs of the partition process are sub-graphs of the DFG, each of which can be mapped to a high-level sub-module in the CGRA top hierarchy. Take the CGRA architecture in Figure 2 as an example: This type of module is a *Core* module. There exists a phenomenon that multiple objectives of the CGRA partition problem may conflict with each other, which means some alterations improve one objective while causing deterioration of other objectives. Pareto improvement is dedicated to solving multi-objective optimization problems. In a Pareto improvement process, solution A is considered superior to B only when there is at least one objective in A that is better than that of B, while the other objectives are not worse than that of B.

Let $G_D = (V, E)$ be the DFG, and $G''_D = (V'', E'')$ is transformed from G_D after the packing process. Nodes in G_D that belong to the same pack are aggregated into one node in G''_D, while the contents in the pack are recorded as attributes of the newly created node. Let I be a k-partition

of DFG G''_D, while $I = \{P_1, \ldots, P_k\}$. $P_1 \cup P_2 \cup \ldots \cup P_k = V''$ and $P_1 \cap P_2 \ldots \cap P_k = \varnothing$. Each $P_i \in I$ must be mapped to a high-level module.

In Figure 2, P_i would then be mapped to a *Core* module. Since *Core* modules are large-size modules, the execution time of $placePart()$ far exceeds that of $placePack()$. Based on the above reason, rather than using the Place sub-tool for feasibility validation as in the packing process, the partition process adopts the following restrictions to guarantee the validity of the outputs.

- *maxUsage:* Each $P_i \in \{P_1, \ldots, P_k\}$ consists of several nodes in G''_D. These nodes can be mapped to low-level modules or units complying with the configuration information and the packing result. A high-level module contains various types of low-level modules. Let these types be $\{T_1, \ldots, T_n\}$. Taking a specific type T_j from $\{T_1, \ldots, T_n\}$. $Usage(T_j, P_i)$ denotes the number of nodes in P_i that are compatible with T_j divided by the number of resources of T_j in a high-level module (in Figure 2, it is a *Core* module). So, $maxUsage$ represents the maximum value that can be reached by $Usage(T_j, P_i)$ traversing all values in its range.
- *maxInputs:* Let $Inputs[i]$ be the number of cut-in edges of P_i in G''_D, and $maxInput$ is the maximum value of $Inputs[i], i \in 1, 2, \ldots, k$.
- *maxOutputs:* Let $Outputs[i]$ be the number of cut-out edges of P_i in G''_D, and $maxOutput$ is the maximum value of $Outputs[i], i \in 1, 2, \ldots, k$.
- *totPorts:* According to the above definitions of $Inputs[i]$ and $Outputs[i]$, $totPorts = \sum_{i=1}^{k}(Inputs[i] + Outputs[i])$.

Some other genetic-algorithm-based partition tools, such as Reference [35], utilize the **deep graph convolution neural network (DGCNN)** to validate the partition results, which is more accurate but slower.

Algorithm 2 describes the optimizing flow of the partition process. The inputs of the partition process are packs and DFG, which are then converted to G''_D. The $initialPOP()$ function in line 1 adopts two methods to generate initial populations. One of the methods is running spectral clustering [10] shortly to produce high-quality partitions concerning the *totPorts*. The Reference [35] partition tool employs the **Kernighan-Lin (KL)** algorithm [33] and DGCNN to initialize the partitions. The KL algorithm may generate low-quality partition results, while the DGCNN needs to be trained before the partition procedure. PANORAMA [57] also utilizes the spectral clustering algorithm, but it directly uses this algorithm to generate the final DFG partition results. The other way is to randomly assign nodes into different parts, which enhances the diversity of the population. After the initial phase, a population with n individuals $\{I_1, \ldots, I_n\}$ is generated.

The crossover operation is inspired by Reference [6], in which a **backbone-based crossover (BBC)** method is employed to generate *Offspring*. BBC is motivated by the observation that high-quality partitions share many groupings of nodes, which are called the backbone. In line 4, we select p individuals from the old population to do crossover and mutation on the selected parents to create new individuals, and each individual is selected by tournament selection strategy: First, we randomly choose several individuals from the old population; then we select the best one into the set $\{I_1, \ldots, I_p\}$ if it is not already present in the set. The proposed crossover in line 5 is designed to preserve the characteristics shared in the parents, and then the $mutate()$ function in line 6 randomly chooses two parts P_i and P_j from *Offspring* $\{P_1, \ldots, P_k\}$ and swaps two nodes $n_i \in P_i, n_j \in P_j$ to perform mutation.

After the previous step, the $paretoOptimal()$ function is called to compare the fitness of the newly generated *Offspring* and each individual in *POP*. The fitness of an individual is calculated by four objectives: $maxUsage$, $maxInputs$, $maxOutputs$, and $totPorts$. Each individual in the *POP*

ALGORITHM 2: GA-partition Based on Pareto Improvement

```
   Input: packed result, DFG G_D
   Output: sub-DFGs
   /* initial population by spectral clustering and random allocation */
1  POP = initialPOP();
2  i = 0;
3  while i++ < iterations do
       /* generate offspring based on backbone crossover */
4      Select p (p ≥ 2) individuals {I_1, ..., I_p} with the tournament selection strategy;
5      Offspring = BBC({I_1, ..., I_p});
6      mutate(Offspring);
       /* update population based on Pareto Improvement and distance */
7      I_Closest = NULL;
8      foreach I in POP do
9          if paretoOptimal (Offspring, I) then
10             if I_Closest == NULL then
11                 I_Closest = I;
12             else if distance(Offspring, I) < distance(Offspring, I_Closest) then
13                 I_Closest = I;
14     if I_Closest ≠ NULL then
15         POP.erase(I_Closest);
16         POP.push(Offspring);
17 rank(POP, weight());
18 return POP.front();
```

is evaluated against the *Offspring* in terms of its fitness, and if that individual is better than the *Offspring*, then its distance from the *Offspring* is calculated subsequently. We use the well-known set-theoretic partition distance [25]: The distance between two individuals I_i and I_j is defined as the minimum number of one-move steps needed to transform one into the other. In the pool of individuals in *POP* that are worse than the *Offspring* in fitness, the one who has the shortest distance to *Offspring* is erased from *POP*, and the *Offspring* is inserted into *POP*. This distance-based population updating strategy contributes to maintaining population diversity. The MOGA evolutionary partition tool [49] exploits the SPEA2 algorithm [61] to update the population, whose convergence speed is limited. Reference [35] utilizes the NSGA-II algorithm [16] in this procedure, but it is not suitable for our 4-dimensional optimization problem.

In line 17, the *weight*() function scores each individual in *POP* according to the weights of $maxUsage$, $maxInputs$, $maxOutputs$, and $totPorts$. The default weight proportion is 1:1:1:1, which is suitable for most occasions. Developers can adjust the weight in the constraint file of Figure 6 according to the CGRA architecture characteristics. For example, if the DFG contains abundant computing operations, then we can increase the weight of $maxUsage$. If there are few interconnection resources between high-level modules, then we can raise the weights of $maxInputs$, $maxOutputs$, and $totPorts$ to optimize the partition process. Finally, the individual with the best score is returned as the optimal result.

4.6 Hierarchical PnR

In HierCGRA, we follow the definition of the mapping problem in Reference [26]. Let G and H be two graphs. A function $f : V(G) \rightarrow V(H)$ is a homomorphism from G to H if any edge $e[u, v]$ of G, we can find a corresponding $e'[f[u], f[v]]$ in H. Many existing DFG mappers, such as

Reference [12], formulate the placement problem as a graph minor problem from the DFG to the RRG/MRRG. In our mapper, the subdivision and smoothing of the graph have been completed by the RRG/MRRG contracting process in Section 4.3 before placement, which removes interconnection nodes from the original RRG/MRRG and replaces them with the edges between the remaining nodes. So, we directly formulate the placement problem in CGRA as the **graph homomorphism (GH)** from the DFG to the contracted RRG/MRRG. Consequently, to ensure the routability of the placement result, we utilize a routing algorithm integrated into the placement process.

The routing algorithm is called frequently when placing nodes from DFG to the contracted RRG/MRRG. Let the $e'[f[u], f[v]]$ be the edge to be routed; the routing problem is defined as finding a path $p(f[u], f[v])$ in the original RRG/MRRG, where the nodes on it consist of only routing resources, including interconnection modules and their ports, and FU with route sharing function for balancing the latencies. Two constraints must be satisfied in the routing problem: (1) Paths with different source nodes do not share overlapping nodes. (2) Signals passing through different paths must reach a sink node on the same cycle. Our CGRA Router is inspired by Pathfinder [41] in FPGA routing and is illustrated in Algorithm 3. The set of edges is traversed sequentially, where each edge is denoted as (u, v). Nodes u and v belong to RRG G_R, which is a directed graph. We first try to find the shortest uncongested path that connects u and v using *findPathBFS*(), which runs a breadth-first search starting from u and ending when reaching v in G_R, while recording the paths traversed. Once the search process encounters a node that breaks the routing constraint, paths from this node will not be retrieved. Simultaneously, paths that reach a duplicate node are trimmed off to prevent the searching process from jamming in loops. If a path is found, then the routing state R' updates and starts to route the next edge. Otherwise, all paths from u to v are sought by *findPaths*(), in which the searching process records the number of conflicting nodes rather than avoid it. The path with the least conflict is selected, while the paths that share overlapping nodes with it are ripped out of R', recorded by *pathsTried* and appended to *edgesToRoute* for re-routing. Paths in *pathsTried* are ignored in *leastConflict*() to avoid repeated attempts at the same path.

ALGORITHM 3: CGRA Router

```
Input: RRG (or MRRG) G_R, routing state R, edges to be route
Output: routing result R'
edgesToRoute = edges;
R' = R;
foreach edge(u, v) in edgesToRoute do
    path = findPathBFS(u, v, G_R);
    if path is found then
        update(R', path);
        continue;
    paths = findPaths(u, v, G_R);
    path = leastConflict(paths, G_R, pathsTried);
    if failed too many times or path is not found then
        return R;
    edgesToReRoute = update(R', path);
    edgesToRoute.append(edgesToReRoute);
    pathsTried.append(edgesToReRoute);
return R';
```

To expedite the path retrieval process and avoid the time-consuming task of repeatedly finding the same paths, we store the frequently used paths in a cache, which allows the *findPaths*() to

swiftly access stored paths when required. Considering HierCGRA's hierarchical architecture, the paths between FUs in the same hierarchy of RRG are stored as a routing library and loaded into memory each time the placement begins, facilitating quick retrieval of paths between any two FUs by fetching memory.

ALGORITHM 4: Graph Homomorphism Mapping Algorithm

```
    Input: DFG G_D, RRG (or MRRG) G_R, compatible Compat, placing state P, routing state R
    Output: placed result P', routed result R'
1   ITL = findUnplaced(G_D, P);
2   Compat = NSPV(ITL, Compat, G_R);
3   P', R' = P, R;
4   index = 0;
5   compatible = Compat[node];
6   compatibleStack = Stack();
7   while index < ITL.length() do
8       node = ITL[index];
9       candidates = prune(node, compatible, G_R, P);
10      traversed = Set();
11      rank(candidates);
12      isSuccess = false;
13      foreach candidate in candidates do
14          traversed.insert(candidate);
15          edgesToRoute = findEdges(candidate, P');
16          isSuccess = route(G_R, R', edgesToRoute);
17          if isSuccess then
18              break;
19      if isSuccess then
20          index += 1;
21          compatible = Compat[node];
22          candidates.erase(traversed);
23          compatibleStack.push(candidate);
24          update(P', R');
25      else
26          index -= 1;
27          compatible = compatibleStack.top();
28          compatibleStack.pop();
29          rollBack(P', R');
30      if index < 0 then
31          return NULL;
32  return P', R';
```

Our GH mapping pseudocode is shown in Algorithm 4. In spatial mapping mode, the algorithm's input of CGRA hardware description is RRG. In temporal mapping mode, this kind of input is MRRG. All the unmapped nodes in DFG are visited and added to the **input traversal list (ITL)** following the given sorting method in line 1, which in our placing algorithm is set to be the search-tree-based scheme from the VF2 algorithm [14] by default. In addition, BFS, DFS, and zigzag from Reference [9] are also available alternatives for DFG sorting. Users can customize their sorting methods or modify them based on existing rules. Nodes with greater degrees are ranked first in the sorting method of this article while following the restriction of STB. To

handle the hardware heterogeneity of our CGRA, a *Compat* table is used to define each DFG node's compatible PEs in line 2. If a PE supports a DFG node's instruction, then it will be in this node's compatibility list. *NSPV* is then adopted for pruning the *Compat* table and reducing the search space, which is particularly effective in hierarchical placing. In the loop, *node* indicates the node currently being mapped, and *candidates* stores the locations in RRG G_R that *node* can be mapped to. The $prune(node, compatible, G_R, P)$ function in line 9 prunes the mapping alternatives by analyzing the current placing situation. In this process, nodes in RRG that have been occupied are erased from candidates, and *1-subgraph-pattern validation* is executed to exclude unmappable positions. After that, *candidates* are ranked according to its adjacency, which in this article is defined as the number of edges between it and the occupied nodes. The thought is based on the fact that the more nodes are scored in adjacency, the more routing resources it has that connect with the mapped part and therefore the easier it is to route them successfully. We then iterate over each option in *candidates* and verify whether the $route(candidates)$ function, illustrated in Algorithm 3, succeeds after mapping *node* to it. Once $route(candidates)$ is successfully executed, we erase the traversed options in *candidates* and push the remaining parts into *compatibleStack* after breaking out of the iteration. Then, the algorithm moves to the next element in ITL while updating P' and R'. If all the candidates failed to satisfy the condition, then we roll back to the previous element in the list and update *candidates* to the set popped out from *compatibleStack* rather than utilize the options stored in *Compat* to try alternatives that were not tested in the last iteration. The GH mapping process returns P' and R' if all elements in ITL are visited. Otherwise, as all the options are tried, the algorithm backtracks to the first *node* in ITL and returns an empty set if $route(candidates)$ function fails.

The GH mapping process is encapsulated as PlacePack, PlacePart, and PlaceTop programs shown in Figure 6. Each of the above programs takes the result of the previous one as the input. We employ a hierarchical placing strategy in which the nodes with more weights are mapped first. In the PlacePart program, nodes of the input DFG can be classified into two types: The ones obtained from the packing process, which should be mapped to a module, and the ones from the original DFG, which should be mapped to a unit. The former weighs more, as there are several nodes of the original DFG in it. Thus, it runs the GH mapping algorithm first, and the latter takes the result of the former as the input. This strategy is proved to be able to accelerate the placing process in experiments, because it maps the hard part first and prunes a considerable number of candidates for the simple part by *NSPV*.

5 ARCHITECTURE EVALUATION AND DSE

The quality of a designed architecture is evaluated in two aspects: ASIC implementation and mapping result. Results of ASIC implementation include area, power consumption, maximum operating frequency, and so on. The mapping results can be evaluated in terms of PE utilization and II. For large-scale benchmarks, mapping success rate and mapping time are also introduced as evaluation criteria. Thanks to the versatility of our graph-based placing and routing algorithm, HierCGRA enables the mapping of heterogeneous architectures and supports manual tuning of constraints to improve the mapping result.

HierCGRA provides flexible architecture design and interconnect patterns, bringing a huge search space. As shown in Figure 2, each FU can be designed for realizing different functions. Simultaneously, the *Block* modules and *Core* modules displayed can also be established with different internal constructs. The interconnections in HierCGRA are realized by switches or GIBs, and they can achieve heterogeneous interconnections by altering the interconnection pattern between FUs and them. This is particularly effective for optimizing architectures with heterogeneous FUs, since the utilization and external connection patterns of different types of functions are usually

inconsistent. HierCGRA supports the automated generation of typical interconnection patterns, such as mesh, diag, and GIB interconnect. The CGRA scale, FU type, and FU distribution are also parameterized. However, complex interconnections are generally difficult to parameterize and often require manual adjustment to accommodate computing domains.

We classify the parameters into the following groups:

— *top:* CGRA scale and the share of each type of FU.
— *computation resource:* The operations supported by each FU.
— *connection resource:* The interconnection pattern between FUs.

Given a function:

$$f(\boldsymbol{x}) \rightarrow \boldsymbol{y}; \boldsymbol{x} \in \boldsymbol{X}, \tag{1}$$

where $\boldsymbol{x}$ represents a set of parameters, $\boldsymbol{X}$ denotes its search space, and $\boldsymbol{y}$ is the result of evaluation. The target of DSE flow is to obtain $\boldsymbol{x}_{opt} \in \boldsymbol{X}$ that yields the optimal value of $\boldsymbol{y}$. Nevertheless, the parsing expressions and working methods in the mapping relation f remain unknown, which means that we can only get the corresponding $\boldsymbol{y}$ value by running the function $f(\boldsymbol{x}) \rightarrow \boldsymbol{y}$. In our CGRA DSE problem, running function $f(\boldsymbol{x})$ means performing an architecture generation with the parameters set as $\boldsymbol{x}$ and implementing evaluation, which is a time-consuming process. The DSE flow can be modeled as a multi-objective black-box optimization problem, and our goal is to find the optimal solution within fewer times running.

HierCGRA's flexible architecture design brings a vast search space. Some existing automatic DSE frameworks, such as REVAMP [4], utilize the SA algorithm to explore the CGRA architecture, while other frameworks, such as RADISH [59], employ the genetic algorithm. They are not effective enough for this situation. To address this challenge, we flatten the CGRA architecture to the level of units and interconnection modules and propose a Bayesian-optimization-based automatic DSE process that contains two phases:

— *Global-DSE:* In this phase, hardware units are fully connected, and all FUs are configured to support comprehensive computing functions, so *computation resource* and *connection resource* parameters are fixed. We explore the design space formed by *top* parameters to find the optimal CGRA scales.
— *Local-DSE:* In this phase, *top* parameters are set to the results of *Global-DSE*, so the CGRA scale is constant. *Computation resource* and *connection resource* parameters construct the design space explored by our architecture optimizer. The target of this step is to find the best heterogeneous FU configuration and connection pattern in a specific CGRA scale.

The CGRA is initialed with a classic regular architecture and constructed with the parameters mentioned above. To evaluate a CGRA architecture, we generate the RRG and assess it by a set of benchmarks to obtain II and Utilization of PE and routing resources. The area of the ASIC implementation is predicted by the number of each type of ALU, multiplexer, and register in the RRG. Thus, the DSE process is a multi-objective optimization problem, which aims to minimize II and area and maximize PE utilization rate simultaneously. In the *Global-DSE* phase, the optimization objective includes hardware area, II, and PE utilization rate. In the *Local-DSE* phase, only area and II are included, for the PE utilization rate remains unchanged in a constant CGRA scale. The above process is defined as a Pareto improvement problem that has been explained in Section 4.5. We employ the **Multiobjective Tree-structured Parzen Estimator (MOTPE)** [45], a variant of the multiobjective Bayesian optimization algorithm with high efficiency for solving the black-box optimization problem to automate the DSE process. The REVAMP [4] framework can only explore heterogeneous hardware configurations on a fixed CGRA scale, while RADISH [59] can not

optimize the *connection resource* parameters. The THRAM [38] DSE framework can explore the global scale and various local heterogeneous hardware parameters of the CGRA, so we compare its performance with our DSE tool in Section 6.3.3.

In our construct of a CGRA architecture, there are multiple parameters with different types and some parameters conflict with each other. MOTPE utilizes tree-structured adaptive Parzen estimators as a surrogate, which naturally handles not only continuous variables but also discrete, categorical, and conditional variables.

Let $\boldsymbol{y}_1$ and $\boldsymbol{y}_2$ be two vectors. We define: ① $\boldsymbol{y}_1 > \boldsymbol{y}_2$, if $\forall i : \boldsymbol{y}_1[i] > \boldsymbol{y}_2[i]$. ② $\boldsymbol{y}_1 \geq \boldsymbol{y}_2$, if $\forall i : \boldsymbol{y}_1[i] \geq \boldsymbol{y}_2[i]$. ③ $\boldsymbol{y}_1 || \boldsymbol{y}_2$, if neither $\boldsymbol{y}_1 \geq \boldsymbol{y}_2$ nor $\boldsymbol{y}_2 \geq \boldsymbol{y}_1$.

Let $\boldsymbol{y}$ be a vector and Y be a set of vectors. We define: ① $\boldsymbol{y} > Y$, if $\exists \boldsymbol{y}_n \in Y : \boldsymbol{y} > \boldsymbol{y}_n$. ② $\boldsymbol{y} \leq Y$, if $\exists \boldsymbol{y}_n \in Y : \boldsymbol{y} \leq \boldsymbol{y}_n$. ③ $\boldsymbol{y} || Y$, if $\forall \boldsymbol{y}_n \in Y : \boldsymbol{y} || \boldsymbol{y}_n$.

Taking Equation (1) as the problem to solve, where both $\boldsymbol{x}$ and $\boldsymbol{y}$ are vectors, we can get a set of observations: $\{(\boldsymbol{x}_1, \boldsymbol{y}_1), (\boldsymbol{x}_2, \boldsymbol{y}_2), \ldots, (\boldsymbol{x}_k, \boldsymbol{y}_k)\}$. MOTPE defines $p(\boldsymbol{x}|\boldsymbol{y})$ as:

$$p(\boldsymbol{x}|\boldsymbol{y}) = \begin{cases} l(\boldsymbol{x}); & \boldsymbol{y} > Y^* \cup \boldsymbol{y} || Y^* \\ g(\boldsymbol{x}); & \boldsymbol{y} \leq Y^* \end{cases}, \tag{2}$$

where Y^* is selected to be a set of vectors to meet the condition: $p(\boldsymbol{y} > Y^* \cup \boldsymbol{y} || Y^*) = \gamma$, and γ is a constant that can be set according to a specific running. $l(\boldsymbol{x})$ is the probability density function formed by the observations that satisfy $\boldsymbol{y} > Y^* \cup \boldsymbol{y} || Y^*$ ($\boldsymbol{y} > Y^*$ or $\boldsymbol{y} || Y^*$), and the probability density function $g(\boldsymbol{x})$ is formed by the rest of the observations.

Expected Hypervolume Improvement (EHVI) [45] is adopted as the acquisition function to guide the optimization direction, which selects candidate $\boldsymbol{x}$ to evaluate. Its derivation process is as follows:

(a) Let $\boldsymbol{r}$ be the reference point, $\lambda(S)$ be the Lebesgue measure of a set S, and $[f_1(\boldsymbol{y}), r_1] \times [f_2(\boldsymbol{y}), r_2] \times \cdots \times [f_m(\boldsymbol{y}), r_m]$ as the m-dimensional hypercuboid consisting of all vectors $\boldsymbol{v}$ that satisfies $\boldsymbol{y} \geq \boldsymbol{v}$ while $\boldsymbol{r} \geq \boldsymbol{v}$ is false. For a vector set Y, its hypervolume indicator $I_H(Y)$ [7] can be obtained by:

$$I_H(Y) := \lambda\left(\bigcup_{\boldsymbol{y} \in Y} [f_1(\boldsymbol{y}), r_1] \times [f_2(\boldsymbol{y}), r_2] \times \cdots \times [f_m(\boldsymbol{y}), r_m]\right). \tag{3}$$

(b) Let C_{Y^*} be a constant obtained by:

$$C_{Y^*} = \int^{R} (I_H(Y^* \cup \{\boldsymbol{y}\}) - I_H(Y^*)) p(\boldsymbol{y}) d\boldsymbol{y}; R = \boldsymbol{y} | \boldsymbol{y} > Y^* \cup \boldsymbol{y} || Y^*. \tag{4}$$

(c) So, EHVI is calculated by the following expression:

$$EHVI_{Y^*} = \frac{C_{Y^*}}{\gamma + (1-\gamma)\frac{g(\boldsymbol{x})}{l(\boldsymbol{x})}}. \tag{5}$$

The framework of the DSE process is presented in Figure 1. Our DSE process does not require a predetermined data set for training. The initial model is generated by random sampling after we set up the design space according to the parameters. Each time a sampling process is completed, MOTPE defines a Gaussian distribution with a specified mean and a standard deviation according to the parameters and evaluation result. These sampling results together form a probability density function $g(\boldsymbol{x})$, which grows closer to the actual distribution $f(\boldsymbol{x})$ as the number of samples

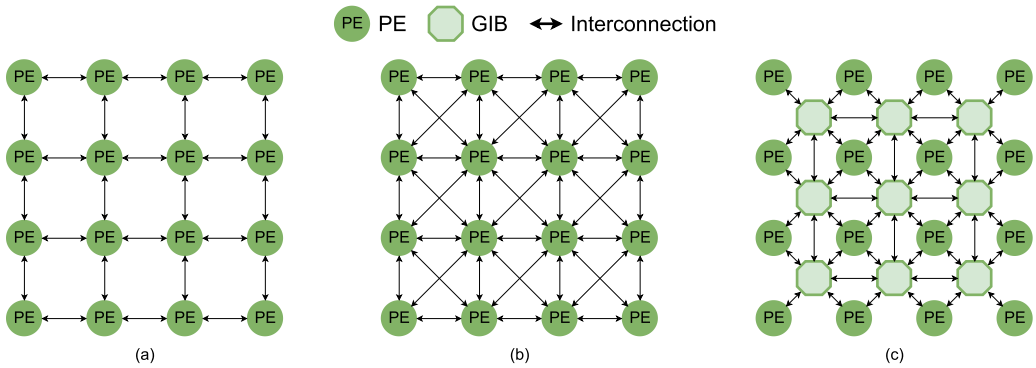

Fig. 10. Different interconnection patterns. (a) The mesh-interconnect pattern. (b) The diag-interconnect pattern. (c) The GIB-interconnect pattern.

Table 3. Area of 4×4 CGRAs with Different Interconnection Patterns

Framework	**CGRA-ME** [2]		**OpenCGRA** [51]	**HierCGRA**			
Connection Pattern	mesh	diag	mesh	mesh	diag	full	GIB
PE Area (μm^2)	10,824	11,840	14,717	9,252	9,252	9,252	9,252
Total Area (μm^2)	188,362	214,902	238,734	185,756	202,145	230,996	199,449
Norm Area	1.014	1.157	1.285	1	1.088	1.244	1.074

increases. And EHVI is employed to select the next point for sampling. This process continues until the termination criteria are satisfied, which in this article refers to the sampling times reaching the limit. Then, the algorithm returns the Pareto solution in the sampled set.

6 EXPERIMENTAL RESULTS

6.1 ASIC Implementation

In this experiment, we construct a CGRA with 4×4 homogeneous PEs. Each PE can handle basic operations, including *add*, *subtract*, *multiply*, *and*, *or*, *xor*, and *shift*. HierCGRA and CGRA-ME [2] are used to model two typical CGRA architectures with distinct interconnection patterns: the mesh-interconnect and the diag-interconnect, which are shown in Figures 10(a)–(b). A mesh-interconnect CGRA architecture modeled by OpenCGRA [51] is also presented for comparison, which consists of a data memory with 4 banks and 4×4 function tiles that support the same functionality as in PE. The mapping performance of each architecture is presented in Section 6.2. To demonstrate the rich design parameters in HierCGRA, we also implement a full-interconnect and a GIB-interconnect CGRA architecture. Each PE can connect with any other PEs in these interconnection patterns. There are no conflicts between any interconnection relationships in the full-interconnect pattern. However, in the GIB-interconnect pattern, these conflicts may occur between some PE pairs, for some shared interconnection resources are limited. Figure 10(c) shows the GIB-interconnect pattern. The RTL code generated by each tool is then fed into Synopsys Design Compiler for logic synthesis. The implementation is based on the SMIC 55nm standard-cell library. The operating frequency is set at 300 MHz with no timing violations occurring.

As shown in Table 3, under the premise of achieving the same connectivity, the area of hardware generated by HierCGRA is reduced by 1.4% compared to CGRA-ME-generated hardware and 22.2% compared to OpenCGRA-generated hardware for the mesh-interconnect architecture. When raising the connectivity to diag-interconnect architecture, the area of CGRA modeled by

Table 4. Mapping Performance of Benchmarks to Different CGRAs

Benchmark				CGRA-ME [2]		OpenCGRA [51]		HierCGRA[1]		HierCGRA[2]	
Name	**IO**	**Node**	**Edge**	**Size**	**II**	**Size**	**II**	**Size**	**II**	**Size**	**II**
accumulate	4	18	22	4×4	2	4×4	5	4×4	2	2×2	4
cap	4	24	29	4×4	3	4×4	5	4×4	3	4×4	2
conv2	3	16	18	4×4	2	4×4	4	2×2	4	2×2	4
conv3	4	24	27	4×4	2	4×4	5	4×4	2	4×4	2
mac	2	11	13	2×2	3	4×4	7	2×2	3	2×2	3
mac2	4	24	30	4×4	3	4×4	9	4×4	3	4×4	2
matrixmultiply	2	17	19	4×4	3	4×4	6	4×4	3	4×4	2
mults1	4	31	35	4×4	5	4×4	5	4×4	4	4×4	2
mults2	4	25	31	4×8	3	4×4	11	4×4	5	4×4	2
simple	3	12	14	4×4	2	4×4	5	2×2	2	2×2	2
simple2	3	12	14	4×4	2	4×4	4	2×2	2	2×2	2
sum	1	7	8	2×2	3	4×4	5	2×2	2	2×2	2
nomem	0	6	7	2×2	2	4×4	4	2×2	2	2×2	2

[1]HierCGRA with mesh-interconnection.
[2]HierCGRA with full-interconnection.

CGRA-ME increased by 14.1%, and that of HierCGRA increased by only 8.8%, which results in a 5.9% area reduction achieved by the latter compared to the former. Even for a HierCGRA-generated full-interconnect architecture, its hardware area is lower than the OpenCGRA-generated mesh-interconnect architecture by 3.2%. The different Verilog generation methods cause this difference. CGRA-ME uses a tool written in C++ to generate the Verilog, while OpenCGRA employs the PyMTL. Our modeling tool utilizes Chisel's Verilog generator to generate the optimized RTL implementation and achieve lower hardware area. Among HierCGRA-generated architectures, the GIB-interconnect architecture's area is lower than the diag-interconnect architecture by 1.3%, but it achieves a high PE connectivity because of the interconnection *tracks* between different GIBs. Table 3 also shows the PE area of HierCGRA-generated architecture is lower than that of CGRA-ME-generated and OpenCGRA-generated ones, and it remains unchanged as the connection pattern varies. This phenomenon is caused by different interconnection architectures. In hardware generated by CGRA-ME and OpenCGRA, the interconnections are implemented by multiplexers or crossbars within PEs. In HierCGRA-generated hardware, interconnection modules (switches and GIBs) are out of the PEs, which can improve the interconnection flexibility.

6.2 Mapping Performance

In mapping tests, we set $N = 6$ as the *NSPV* parameter. The parameter is explained in Section 4.3.

6.2.1 PE Utilization. The flexible interconnection in HierCGRA poses a challenge for the mapping algorithm. We perform mapping on each of the three mesh-interconnect architectures generated by CGRA-ME, OpenCGRA, and HierCGRA. Their own mappers are used to measure the mapping effectiveness, with all settings following the frameworks' defaults. A full-interconnect architecture is also tested to demonstrate the effectiveness of HierCGRA's flexible interconnection. We first conduct our experiments with the scale set at 4×4, and if the obtained II equals 1, we shrink the scale to 2×2 to prevent the mapper from not reaching the limit of its capacity because of abundant resources. For benchmark *mults2*, we expand the scale of CGRA-ME to 4×8 and double its IO units, because it failed to map as it exceeds the maximum specified by its mapper. Table 4 shows the result of the mapping performance to different CGRA. Let the total number of nodes

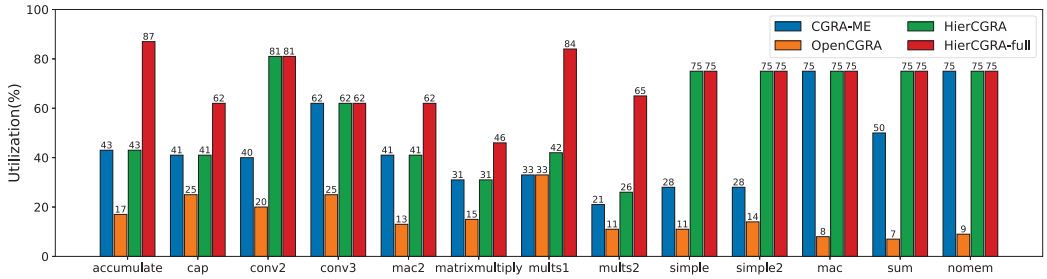

Fig. 11. PE utilization of benchmarks to different CGRAs.

in the DFG be n_{total} and n_{io} be the number of nodes that have to be mapped to the IO ports of CGRA, and n_{PEs} represents the number of PEs. We calculate the utilization by Equation (6) and show the result in Figure 11. All the benchmarks are from CGRA-ME.

$$Util = \frac{n_{total} - n_{io}}{n_{PEs} \times II} \tag{6}$$

In all 13 cases, HierCGRA achieves the highest utilization. The average utilization of HierCGRA is 57.1%, while that of OpenCGRA and CGRA-ME are 15.9% and 43.7%, respectively. By Adding Interconnection to HierCGRA, utilization could be further increased. By extending interconnection flexibility to full-connection, HierCGRA can achieve an average utilization of 71.1% and a maximum of 87.0% while still requiring less area than CGRA-ME and OpenCGRA as shown in Table 3.

6.2.2 Mapping Time. In this section, we compare our GH-based mapper with the ILP mapping algorithm from CGRA-ME and the RAMP algorithm from CCF [15]. CGRA-ME also offers an SA-based solution, but its performance on all benchmarks is inferior to that of ILP. Therefore, we build an SA mapper based on HierCGRA as a comparison. OpenCGRA mapper is not included, since it is architecture-bound and prefers to find feasible solutions while lacking optimization. A 4×4 ADRES CGRA with extra toroid connections is constructed using the description format of each compiler to eliminate the impact of architecture. We measure the execution time using an i7-12700 CPU @ 3.60 GHz and restrict the number of threads in use to 8. In all 13 benchmarks, the GH-based mapper achieves the best II, the same as ILP, but takes less time, as is present in Figure 12. On average, our GH-based algorithm runs 41 times faster than ILP, 54 times faster than RAMP, and 23 times faster than SA.

6.2.3 Hierarchical Mapping Experiment. To verify HierCGRA's hierarchical mapping performance on large-scale CGRA and benchmarks, we build a large-scale CGRA with the modeling template in Section 3. The architecture of the CGRA is shown in Figure 13. There are 16 *Block* modules in one *Core* module, which can be divided into three types: *ALB Block* module, *ARI Block* module, and *MEB Block* module. These three types of modules consist of general-purpose PE, specialized arithmetic units such as multiplication and division, and register units, respectively. In the top hierarchy, four *Core* modules and several IO units are present, and the number of *Core* modules in use can be restricted according to the DFG's scale to raise utilization. The final architecture contains 224 heterogeneous FUs, yielding an RRG with 32,572 nodes and 71,772 edges. We select 7 cases from Express [44] with operands greater than 100 and set the execution environment to be the same as Section 6.2.2.

Table 5 presents the experimental results. We adjust the number of activated CGRA *Core* modules in the hardware architecture to fit the scale of each benchmark and every type of operator

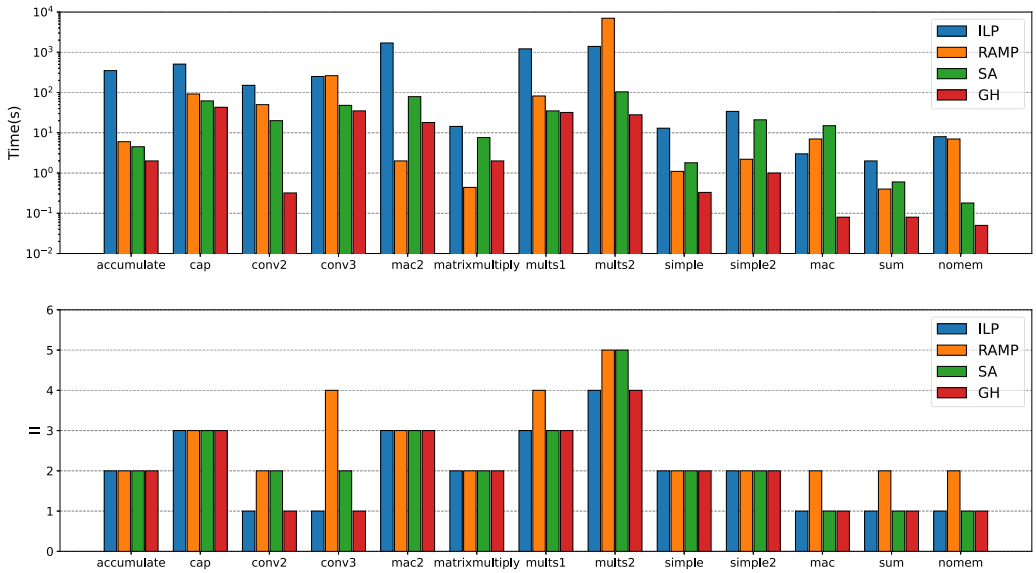

Fig. 12. Execution time and II of different mappers.

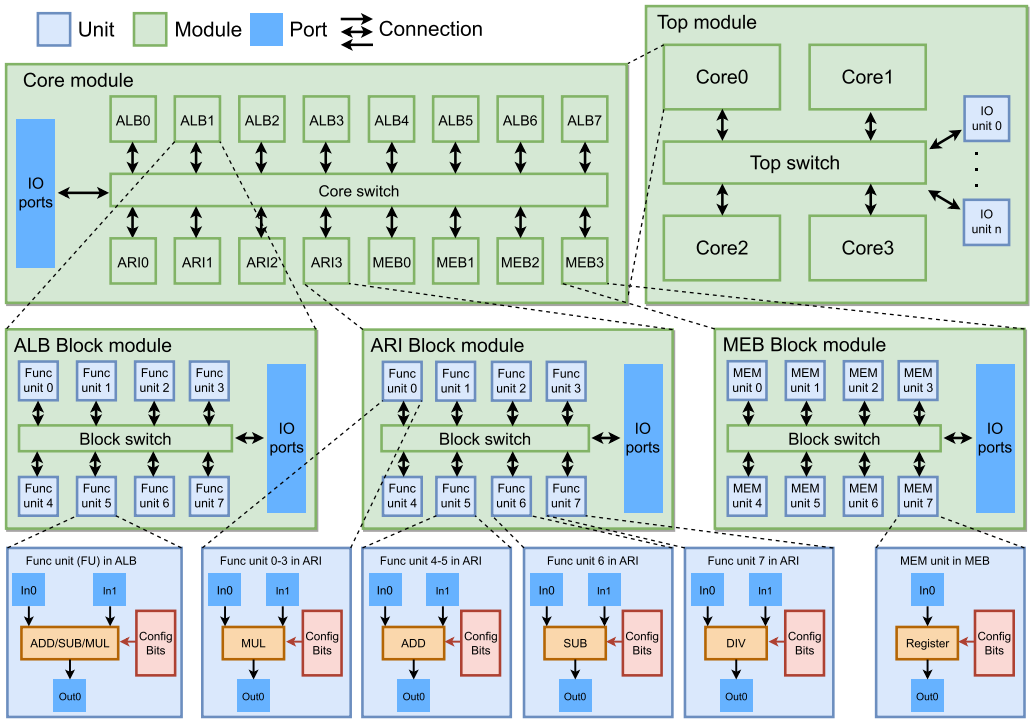

Fig. 13. The architecture of the CGRA used in the hierarchical mapping experiment.

Table 5. Mapping Performance of Benchmarks with Different Strategies

Benchmark				Mapping Time (s)			
Name	**Node**	**Edge**	**Core**	**SA**[1]	**GH**[2]	**GH**[3]	**Hierarchical Mapping**[4]
bmp	106	88	4	2,733.5	790.0	146.2	134.2
cosine1	134	169	1	2,147.0	>7,200	297.3	219.3
cosine2	122	162	2	1,034.4	>7,200	>7,200	348.4
interpolate	108	104	4	>7,200	>7,200	70.1	68.1
matmul	109	116	2	>7,200	>7,200	>7,200	597.7
matinv	333	354	4	>7,200	>7,200	297.4	95.2
smooth	197	196	4	>7,200	>7,200	460.6	189.2

[1]SA Algorithm with Packing and Partition.
[2]The Original GH Algorithm.
[3]GH Algorithm with Packing and Partition.
[4]GH Algorithm with Packing, Partition, and *NSPV*.

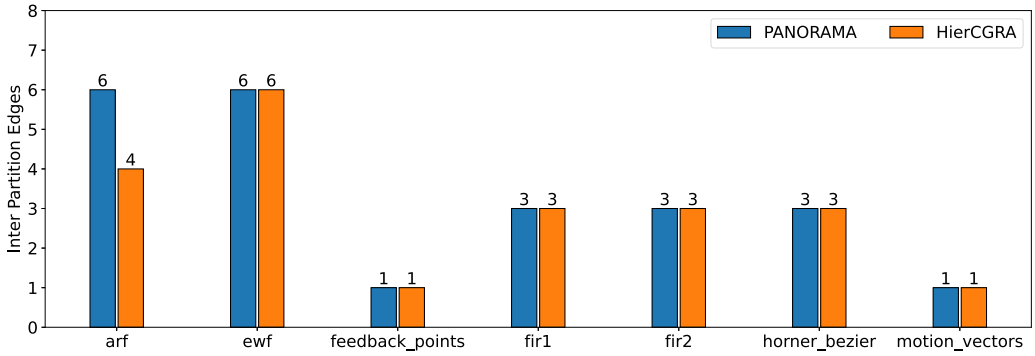

Fig. 14. Partition comparison of PANORAMA and HierCGRA (lower is better).

in it to optimize resource utilization and measure the performance of mapping algorithms. The runtime limitation for each program is set as 7,200 s, and once it exceeds this time, we consider it as failed to map. Our hierarchical mapping algorithm is an extension version of graph homomorphism, with Packing, Partition, and *NSPV* embedded in it. To demonstrate the effectiveness of the hierarchical strategy, we examine the performance of the original GH algorithm and GH algorithm with packing and partition. The SA algorithm with packing and partition is set as the control group while the data of the original SA algorithm is not presented in Table 5, since it exceeds the time limit in all benchmarks.

Experimental results show that our hierarchical mapping algorithm achieves a speedup of at least 26.6× than the Original GH Algorithm. Comparing the result of the GH algorithm (with Packing and Partition) with the hierarchical mapping algorithm, we can conclude that *NSPV* achieves a speedup of at least 9.5×. This can be attributed to the heterogeneous architecture design and the efficient pruning mechanism of *NSPV*. In addition, the experimental results confirm the role of the Packing and Partition algorithm in reducing the search space to accelerate the mapping process.

We also compare HierCGRA's DFG partition quality with PANORAMA [57], and the results are presented in Figure 14. To make a fair comparison with PANORAMA, we decouple our DFG partition algorithm from architecture-aware procedures, such as DFG packing. We implement a new initial population generation algorithm in our partition tool instead of using the spectral

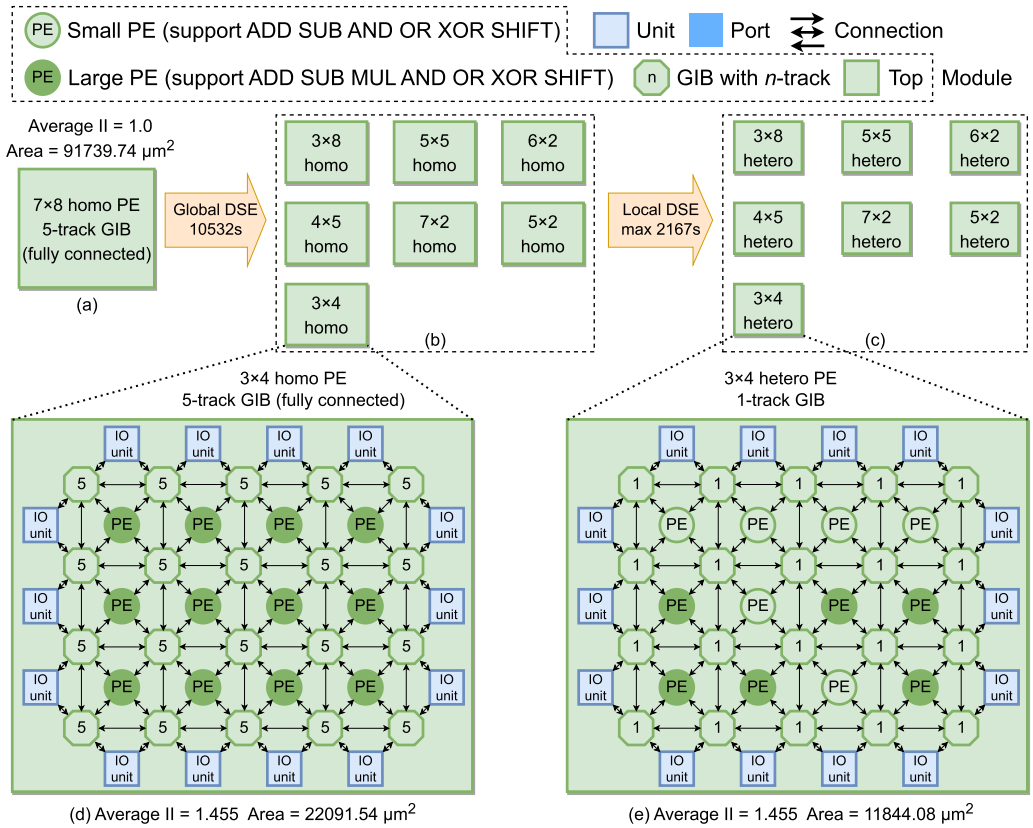

Fig. 15. The configuration of CGRAs in DSE process. (a) The initial CGRA hardware. (b) The homo-CGRAs found by *Global-DSE*. (c) The hetero-CGRAs found by *Local-DSE*. (d) The 3×4 homo-CGRA architecture. (e) The 3×4 hetero-CGRA architecture.

clustering algorithm or randomly distributing the nodes. The partition number is set to 4. Several Express [44] benchmarks commonly used in CGRA DFG mapping are tested. Our mapper achieves similar DFG partition quality with PANORAMA in the tests.

6.3 DSE Results

We construct a CGRA with GIB connection by HierCGRA and apply our DSE algorithm to enhance its performance, which is quantified by area, PE utilization rate, and II. The DSE process is shown in Figure 15. The CGRA has input and output ports in all directions, which provides sufficient IO resources. MOTPE is employed to automate the DSE process, and a fully homogeneous-GIB-interconnect architecture is set as the baseline.

6.3.1 Area Prediction. The hardware area (on ASAP 7nm [13]) is estimated by the architecture modeling system, for the conventional logic-synthesizing-based area calculation process is time-consuming. The area estimator detects the type and configuration parameters of CGRA hardware units and adds up their pre-synthesized area obtained on the ASAP 7nm [13] standard-cell library using Synopsys **Design Compiler (DC)**. To verify its accuracy, we compare the CGRA's estimated area with the DC synthesized area in different hardware scales. The test result is in Figure 16(a), which shows the area estimation error is quite low (max 1.85% in 2×2 CGRA scale). For larger

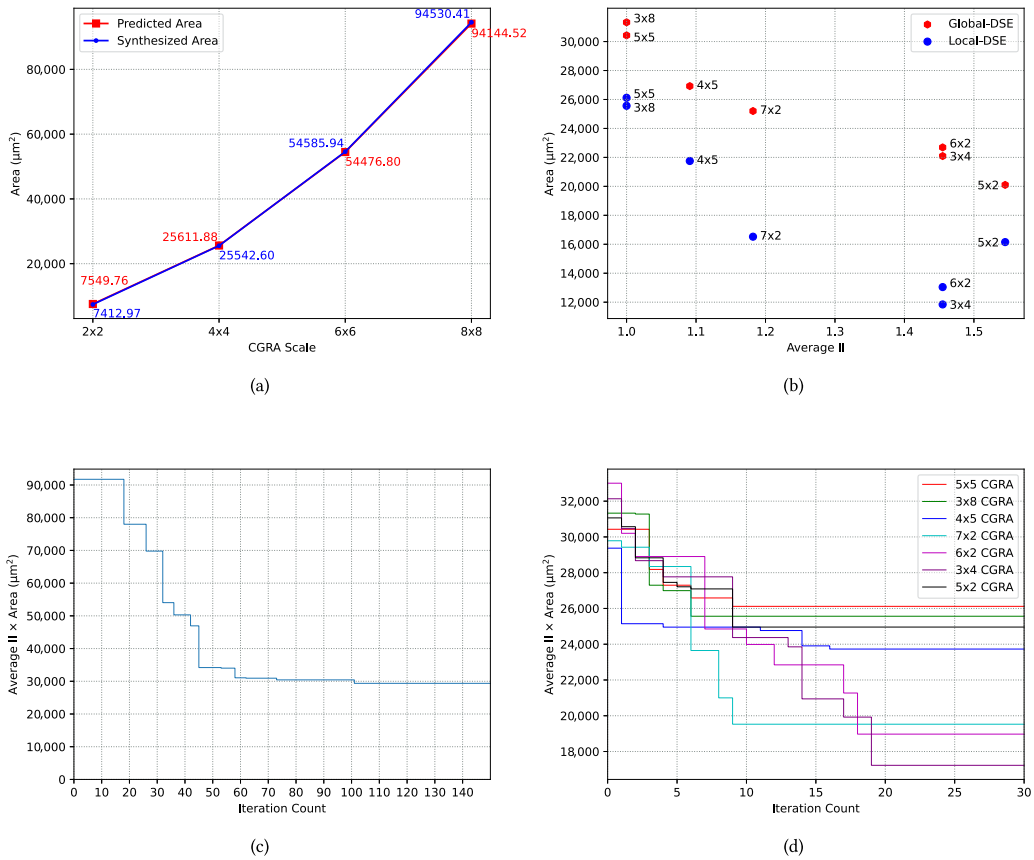

Fig. 16. DSE performance. (a) The accuracy of hardware area prediction system. (b) Area and average II comparisons of *Global-DSE* and *Local-DSE*. (c) The optimal average II-area product in *Global-DSE* process. (d) The optimal average II-area product in *Local-DSE* process.

CGRAs, the estimation error is lower than 0.5%. All tested CGRAs are GIB-connected with maximum connection complexity configuration, and the GIB track number is set to 4.

6.3.2 Hardware Utilization Improvement. PE utilization rate and II are measured by a group of selected medium-size benchmarks, which include several frequently used applications and some relatively large-scale CGRA-ME benchmarks. The benchmarks and PE utilization results after DSE are shown in Figure 17. Compared to CGRA-ME and OpenCGRA, our optimized CGRA hardware reaches a higher PE utilization rate in the benchmarks. And the 5×2 CGRA keeps the utilization rate ≥70% in all benchmarks. As the CGRA scale reduces, some applications' PE utilization rate keeps improving, while others' utilization rate begins to fall after reaching a peak, for they need higher II to implement their computing functions.

6.3.3 Exploration Performance. Figure 15(a) and Figure 15(b) show the hardware scale changes in the *Global-DSE* process. In this phase, several optimal CGRA scales on the Pareto front are found. As shown in Figure 16(c), during the *Global-DSE* process, the average II-area product of the architecture obtained after 150 iterations is reduced by 69.8% compared to the beginning, and the optimal result is reached in the 101st iteration. The running time of *Global-DSE* is 10,532 seconds. The *Local-DSE* process optimizes the PE functions and connection resources of CGRAs found

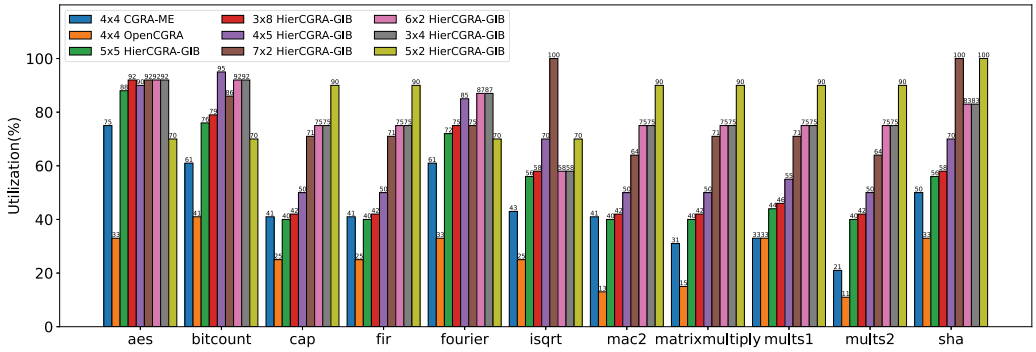

Fig. 17. PE utilization rate of benchmarks after DSE.

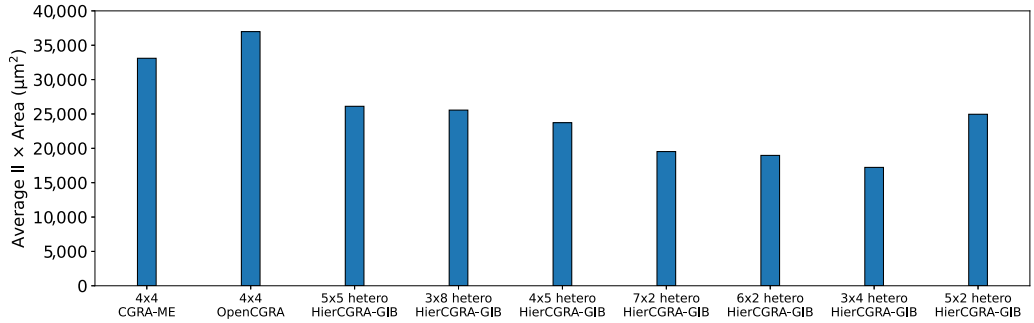

Fig. 18. Average II-area product of different CGRAs.

in the *Global-DSE* process. Figure 15(d) and Figure 15(e) show the hardware changes of the 3×4 CGRA in the *Local-DSE* process. To reduce the hardware area, the GIB track number is cut down, and some PEs' multiplication function support is pruned while others are still capable of comprehensive instructions. Figure 16(d) shows the average II-area product trends during *Local-DSE* phases. Each *Local-DSE* process iterates 30 times, and all of them converge before the 21st iteration. The longest running time of *Local-DSE* is 2,167 seconds, which is the 7×2 CGRA's. Figure 16(b) indicates the area and average II in homogeneous *Global-DSE* and heterogeneous *Local-DSE* processes. Reaching the same average II with homogeneous architecture is included in the selection standard of optimal *Local-DSE* results. Notably, the solution obtained by the *Local-DSE* process realizes a significant average II-area product reduction, which varies from 14.16% to 46.39%, while maintaining the same average II with the homo-PE architecture. The most obvious average II-area product reductions occur in the 6×2 and 3×4 CGRA architecture. Larger CGRAs have numerous hardware units, so their hardware area is hard to reduce, while the average II reaches close to 1. For smaller CGRAs, their hardware units are too few, so the substantial reduction of connections and PE functions can easily cause application mapping failure. Figure 18 compares the average II-area product of different CGRAs generated by CGRA-ME, OpenCGRA, and our HierCGRA DSE tool. Compared with the CGRA-ME-generated hardware, the CGRAs found by our DSE process achieve 21.1%–47.9% lower average II-area product. Compared with the OpenCGRA-generated hardware, they accomplish 29.4%–53.4% lower average II-area product. The best CGRA hardware architecture obtained in DSE is 3×4 heterogeneous CGRA. This CGRA's average II in benchmarks is 1.455, while its area is below 12000μm^2, which is shown in Figure 15(e). Except for the *isqrt*

Table 6. Comparison of Different CGRA DSE Frameworks

DSE Stage	baseline	HierCGRA homo	HierCGRA hetero	THRAM [38] homo	THRAM [38] hetero
CGRA Scale	7×8	3×4	3×4	2×6	2×6
Area/μm^2	91,739.74	22,091.54	11,844.08	24,263.69	13,226.50
Average II	1.0	1.455	1.455	1.455	1.455
Average II × Area	91,739.74	32,143.19	17,233.14	35,303.67	19,244.56
Average II × Area Relative Ratio	1.0×	0.35×	0.19×	0.38×	0.21×

application (58% utilization rate), this CGRA architecture's PE utilization rate is ≥75% in other benchmarks.

We compare the DSE performance of HierCGRA with THRAM [38]. Benchmarks in Section 6.3.2 are used for the test. We use the CGRA in Figure 15(a) as the baseline hardware of THRAM and our framework. The results of DSE performance are shown in Table 6. The hardware found by HierCGRA DSE achieves the minimum average II-area product in the test. Compared with the THRAM framework, HierCGRA DSE can achieve an 8.95% lower average II-area product when finding the optimal homogeneous CGRA scale (*Global-DSE*) and accomplish a 10.45% lower average II-area product when finding the optimal heterogeneous hardware parameters (*Local-DSE*).

However, our current DSE process is time-consuming. Though our area prediction model can avoid frequently running the ASIC implementation flow and save time, the DSE process can still take hours to run a set of large-scale DFG mapping tests repeatedly. Moreover, our DSE tool can only explore the flattened CGRA architecture. The fast automated DSE algorithm for hierarchical heterogeneous CGRA is left as future work.

7 CONCLUSION

In this article, we propose HierCGRA, a novel framework that integrates CGRA modeling, LLVM-based DFG generation, DFG mapping, Verilog generation, architecture evaluation, and DSE. The hierarchical modeling approach and the corresponding mapping algorithm based on graph homomorphism enable the rapid development of large-scale heterogeneous CGRA. Furthermore, the flexible interconnection model and the DSE framework developed on Bayesian optimization pave the way for the design exploration of CGRA architecture. The experimental results show that HierCGRA achieves improved performance compared to existing state-of-the-art platforms in terms of PE utilization, mapping speed, II, and area while providing advanced scalability in modeling and mapping of large-scale heterogeneous CGRAs. The faster DSE flow for hierarchical CGRA architectures is reserved for our future work. HierCGRA's DFG mapper is available at https://gitlab.com/oceancove/hiercgra

REFERENCES

[1] Ensieh Aliagha and Diana Göhringer. 2022. Energy efficient design of coarse-grained reconfigurable architectures: Insights, trends and challenges. In *International Conference on Field-Programmable Technology (ICFPT'22)*. 1–11. DOI: https://doi.org/10.1109/ICFPT56656.2022.9974339

[2] Jason Anderson, Rami Beidas, Vimal Chacko, Hsuan Hsiao, Xiaoyi Ling, Omar Ragheb, Xinyuan Wang, and Tianyi Yu. 2021. CGRA-ME: An open-source framework for CGRA architecture and CAD research. In *IEEE 32nd International Conference on Application-specific Systems, Architectures and Processors (ASAP'21)*. 156–162. DOI: https://doi.org/10.1109/ASAP52443.2021.00030

[3] Mahesh Balasubramanian and Aviral Shrivastava. 2022. PathSeeker: A fast mapping algorithm for CGRAs. In *Design, Automation & Test in Europe Conference & Exhibition (DATE'22)*. IEEE, 268–273.

[4] Thilini Kaushalya Bandara, Dhananjaya Wijerathne, Tulika Mitra, and Li-Shiuan Peh. 2022. REVAMP: A systematic framework for heterogeneous CGRA realization. In *27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'22)*. Association for Computing Machinery, New York, NY, 918–932. DOI: https://doi.org/10.1145/3503222.3507772

[5] Una Benlic and Jin-Kao Hao. 2011. An effective multilevel tabu search approach for balanced graph partitioning. *Comput. Operat. Res.* 38, 7 (2011), 1066–1075.

[6] Una Benlic and Jin-Kao Hao. 2011. A multilevel memetic approach for improving graph k-partitions. *IEEE Trans. Evolut. Computat.* 15, 5 (2011), 624–642.

[7] Dimo Brockhoff, Tobias Friedrich, and Frank Neumann. 2008. Analyzing hypervolume indicator based algorithms. In *10th International Conference on Parallel Problem Solving from Nature (PPSN'08)*. Springer, 651–660.

[8] Stephen P. Brooks and Byron J. T. Morgan. 1995. Optimization using simulated annealing. *J. R. Stat. Societ.: Series D (Statistic.)* 44, 2 (1995), 241–257.

[9] Michael Canesche, Marcelo Menezes, Westerley Carvalho, Frank Sill Torres, Peter Jamieson, José Augusto Nacif, and Ricardo Ferreira. 2020. Traversal: A fast and adaptive graph-based placement and routing for CGRAs. *IEEE Trans. Comput.-aid. Des. Integ. Circ. Syst.* 40, 8 (2020), 1600–1612.

[10] Pak K. Chan, Martine D. F. Schlag, and Jason Y. Zien. 1993. Spectral k-way ratio-cut partitioning and clustering. In *30th international Design Automation Conference*. 749–754.

[11] George Charitopoulos, Ioannis Papaefstathiou, and Dionisios N. Pnevmatikatos. 2021. Creating customized CGRAs for scientific applications. *Electronics* 10, 4 (2021). DOI: https://doi.org/10.3390/electronics10040445

[12] Liang Chen and Tulika Mitra. 2014. Graph minor approach for application mapping on CGRAs. *ACM Trans. Reconfig. Technol. Syst.* 7, 3 (2014), 1–25.

[13] Lawrence T. Clark, Vinay Vashishtha, Lucian Shifren, Aditya Gujja, Saurabh Sinha, Brian Cline, Chandarasekaran Ramamurthy, and Greg Yeric. 2016. ASAP7: A 7-nm finFET predictive process design kit. *Microelectron. J.* 53 (2016), 105–115.

[14] Luigi P. Cordella, Pasquale Foggia, Carlo Sansone, and Mario Vento. 2004. A (sub) graph isomorphism algorithm for matching large graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 26, 10 (2004), 1367–1372.

[15] Shail Dave, Mahesh Balasubramanian, and Aviral Shrivastava. 2018. RAMP: Resource-aware mapping for CGRAs. In *55th ACM/ESDA/IEEE Design Automation Conference (DAC'18)*. 1–6. DOI: https://doi.org/10.1109/DAC.2018.8465892

[16] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan. 2002. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Trans. Evolut. Computat.* 6, 2 (2002), 182–197. DOI: https://doi.org/10.1109/4235.996017

[17] Caleb Donovick, Makai Mann, Clark Barrett, and Pat Hanrahan. 2019. Agile SMT-based mapping for CGRAs with restricted routing networks. In *International Conference on ReConFigurable Computing and FPGAs (ReConFig'19)*. IEEE, 1–8.

[18] Yiran Du, Wei Li, Zibin Dai, and Longmei Nan. 2020. PVHArray: An energy-efficient reconfigurable cryptographic logic array with intelligent mapping. *IEEE Trans. Very Large Scale Integ. Syst.* 28, 5 (2020), 1302–1315.

[19] Murali Emani, Venkatram Vishwanath, Corey Adams, Michael E. Papka, Rick Stevens, Laura Florescu, Sumti Jairath, William Liu, Tejas Nama, and Arvind Sujeeth. 2021. Accelerating scientific applications with SambaNova reconfigurable dataflow architecture. *Comput. Sci. Eng.* 23, 2 (2021), 114–119.

[20] Hadi Esmaeilzadeh, Emily Blem, Renee St. Amant, Karthikeyan Sankaralingam, and Doug Burger. 2011. Dark silicon and the end of multicore scaling. In *38th Annual International Symposium on Computer Architecture (ISCA'11)*. Association for Computing Machinery, New York, NY, 365–376. DOI: https://doi.org/10.1145/2000064.2000108

[21] Xitian Fan, Huimin Li, Wei Cao, and Lingli Wang. 2016. DT-CGRA: Dual-track coarse-grained reconfigurable architecture for stream applications. In *26th International Conference on Field Programmable Logic and Applications (FPL'16)*. IEEE, 1–9.

[22] Stephen Friedman, Allan Carroll, Brian Van Essen, Benjamin Ylvisaker, Carl Ebeling, and Scott Hauck. 2009. SPR: An architecture-adaptive CGRA mapping tool. In *ACM/SIGDA International Symposium on Field Programmable Gate Arrays*. 191–200.

[23] Tong Geng, Chunshu Wu, Cheng Tan, Bo Fang, Ang Li, and Martin Herbordt. 2020. CQNN: A CGRA-based QNN Framework. In *IEEE High Performance Extreme Computing Conference (HPEC'20)*. 1–7. DOI: https://doi.org/10.1109/HPEC43674.2020.9286194

[24] Yijiang Guo, Jiarui Wang, Jiaxi Zhang, and Guojie Luo. 2021. Formulating data-arrival synchronizers in integer linear programming for CGRA mapping. In *58th ACM/IEEE Design Automation Conference (DAC'21)*. 943–948. DOI: https://doi.org/10.1109/DAC18074.2021.9586267

[25] Dan Gusfield. 2002. Partition-distance: A problem and class of perfect graphs arising in clustering. *Inf. Process. Lett.* 82, 3 (2002), 159–164.

[26] Geňa Hahn and Claude Tardif. 1997. Graph homomorphisms: Structure and symmetry. In *Graph Symmetry*. Springer, 107–166.

[27] Mahdi Hamzeh, Aviral Shrivastava, and Sarma Vrudhula. 2012. EPIMap: Using epimorphism to map applications on CGRAs. In *49th Annual Design Automation Conference*. 1284–1291.

[28] Mahdi Hamzeh, Aviral Shrivastava, and Sarma Vrudhula. 2013. REGIMap: Register-aware application mapping on coarse-grained reconfigurable architectures (CGRAs). In *50th Annual Design Automation Conference*. 1–10.

[29] John L. Hennessy and David A. Patterson. 2019. A new golden age for computer architecture. *Commun. ACM* 62, 2 (2019), 48–60.

[30] Li Jiao, Cheng Luo, Wei Cao, Xuegong Zhou, and Lingli Wang. 2017. Accelerating low bit-width convolutional neural networks with embedded FPGA. In *27th International Conference on Field Programmable Logic and Applications (FPL'17)*. IEEE, 1–4.

[31] Manupa Karunaratne, Aditi Kulkarni Mohite, Tulika Mitra, and Li-Shiuan Peh. 2017. HyCUBE: A CGRA with reconfigurable single-cycle multi-hop interconnect. In *54th Annual Design Automation Conference*. 1–6.

[32] Reza Kazerooni-Zand, Mehdi Kamal, Ali Afzali-Kusha, and Massoud Pedram. 2023. Memristive-based mixed-signal CGRA for accelerating deep neural network inference. *ACM Trans. Des. Autom. Electron. Syst.* (May 2023). DOI: https://doi.org/10.1145/3595638

[33] B. W. Kernighan and S. Lin. 1970. An efficient heuristic procedure for partitioning graphs. *Bell Syst. Technic. J.* 49, 2 (1970), 291–307. DOI: https://doi.org/10.1002/j.1538-7305.1970.tb01770.x

[34] Takuya Kojima, Boma Adhi, Carlos Cortes, Yiyu Tan, and Kentaro Sano. 2022. An architecture- independent CGRA compiler enabling OpenMP applications. In *IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW'22)*. 631–638. DOI: https://doi.org/10.1109/IPDPSW55747.2022.00112

[35] Takuya Kojima, Ayaka Ohwada, and Hideharu Amano. 2022. Mapping-aware kernel partitioning method for CGRAs assisted by deep learning. *IEEE Trans. Parallel Distrib. Syst.* 33, 5 (2022), 1213–1230. DOI: https://doi.org/10.1109/TPDS.2021.3107746

[36] Kalhan Koul, Jackson Melchert, Kavya Sreedhar, Leonard Truong, Gedeon Nyengele, Keyi Zhang, Qiaoyi Liu, Jeff Setter, Po-Han Chen, Yuchen Mei, Maxwell Strange, Ross Daly, Caleb Donovick, Alex Carsello, Taeyoung Kong, Kathleen Feng, Dillon Huff, Ankita Nayak, Rajsekhar Setaluri, James Thomas, Nikhil Bhagdikar, David Durst, Zachary Myers, Nestan Tsiskaridze, Stephen Richardson, Rick Bahr, Kayvon Fatahalian, Pat Hanrahan, Clark Barrett, Mark Horowitz, Christopher Torng, Fredrik Kjolstad, and Priyanka Raina. 2023. AHA: An agile approach to the design of coarse-grained reconfigurable accelerators and compilers. *ACM Trans. Embed. Comput. Syst.* 22, 2, Article 35 (Jan. 2023), 34 pages. DOI: https://doi.org/10.1145/3534933

[37] C. Lattner and V. Adve. 2004. LLVM: A compilation framework for lifelong program analysis & transformation. In *International Symposium on Code Generation and Optimization (CGO'04)*. 75–86. DOI: https://doi.org/10.1109/CGO.2004.1281665

[38] Jingyuan Li, Yunhui Qiu, Guowei Zhu, Qilong Zhu, Wenbo Yin, and Lingli Wang. 2023. THRAM: A template-based heterogeneous CGRA modeling framework supporting fast DSE. In *IEEE International Symposium on Circuits and Systems (ISCAS'23)*. DOI: https://doi.org/10.1109/ISCAS46773.2023.10182204

[39] Leibo Liu, Jianfeng Zhu, Zhaoshi Li, Yanan Lu, Yangdong Deng, Jie Han, Shouyi Yin, and Shaojun Wei. 2019. A survey of coarse-grained reconfigurable architecture and design: Taxonomy, challenges, and applications. *ACM Comput. Surv.* 52, 6 (2019), 1–39.

[40] Xingchen Man, Jianfeng Zhu, Guihuan Song, Shouyi Yin, Shaojun Wei, and Leibo Liu. 2022. CaSMap: Agile mapper for reconfigurable spatial architectures by automatically clustering intermediate representations and scattering mapping process. In *49th Annual International Symposium on Computer Architecture*. 259–273.

[41] Larry McMurchie and Carl Ebeling. 1995. PathFinder: A negotiation-based performance-driven router for FPGAs. In *ACM 3rd International Symposium on Field-programmable Gate Arrays*. 111–117.

[42] Bingfeng Mei, Serge Vernalde, Diederik Verkest, Hugo De Man, and Rudy Lauwereins. 2003. ADRES: An architecture with tightly coupled VLIW processor and coarse-grained reconfigurable matrix. In *International Conference on Field Programmable Logic and Applications*. Springer, 61–70.

[43] Kevin E. Murray, Oleg Petelin, Sheng Zhong, Jia Min Wang, and Vaughn Betz. 2020. VTR 8: High-performance CAD and customizable FPGA architecture modelling. *ACM Trans. Reconfig. Technol. Syst.* 13, 2 (2020), 1–55.

[44] S. B. University of California. 2021. EXPRESS Benchmarks. *Website*. Retrieved from: https://web.ece.ucsb.edu/EXPRESS/benchmark/

[45] Yoshihiko Ozaki, Yuki Tanigaki, Shuhei Watanabe, and Masaki Onishi. 2020. Multiobjective tree-structured Parzen estimator for computationally expensive optimization problems. In *Genetic and Evolutionary Computation Conference*. 533–541.

[46] Y. Qiu, Y. Cao, Y. Dai, W. Yin, and L. Wang. 2022. TRAM: An open-source template-based reconfigurable architecture modeling framework. In *32nd International Conference on Field-Programmable Logic and Applications (FPL'22)*. IEEE Computer Society, 61–69. DOI: https://doi.org/10.1109/FPL57034.2022.00021

[47] Tongzhou Qu, Zibin Dai, Yanjiang Liu, and Lin Chen. 2022. A high flexible shift transformation unit design approach for coarse-grained reconfigurable cryptographic arrays. *Electronics* 11, 19 (2022). DOI: https://doi.org/10.3390/electronics11193144

[48] Gokhan Sayilar and Derek Chiou. 2014. Cryptoraptor: High throughput reconfigurable cryptographic processor. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD'14)*. IEEE, 155–161.

[49] Weiguang Sheng, Weifeng He, Jianfei Jiang, and Zhigang Mao. 2012. Pareto optimal temporal partition methodology for reconfigurable architectures based on multi-objective genetic algorithm. In *IEEE 26th International Parallel and Distributed Processing Symposium Workshops & PhD Forum*. 425–430. DOI: https://doi.org/10.1109/IPDPSW.2012.54

[50] Kaichuang Shi, Xuegong Zhou, Hao Zhou, and Lingli Wang. 2022. An optimized GIB routing architecture with bent wires for FPGA. *ACM Trans. Reconfigurable Technol. Syst.* 16, 1, Article 2 (Dec. 2022), 28 pages. DOI: https://doi.org/10.1145/3519599

[51] Cheng Tan, Nicolas Bohm Agostini, Jeff Zhang, Marco Minutoli, Vito Giovanni Castellana, Chenhao Xie, Tong Geng, Ang Li, Kevin Barker, and Antonino Tumeo. 2021. OpenCGRA: Democratizing coarse-grained reconfigurable arrays. In *IEEE 32nd International Conference on Application-specific Systems, Architectures and Processors (ASAP'21)*. 149–155. DOI: https://doi.org/10.1109/ASAP52443.2021.00029

[52] Christopher Torng, Peitian Pan, Yanghui Ou, Cheng Tan, and Christopher Batten. 2021. Ultra-elastic CGRAs for irregular loop specialization. In *IEEE International Symposium on High-Performance Computer Architecture (HPCA'21)*. 412–425. DOI: https://doi.org/10.1109/HPCA51647.2021.00042

[53] Fengbin Tu, Weiwei Wu, Shouyi Yin, Leibo Liu, and Shaojun Wei. 2018. RANA: Towards efficient neural acceleration with refresh-optimized embedded DRAM. In *ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA'18)*. IEEE, 340–352.

[54] University of California Berkeley. 2020. Chisel-tester2. Retrieved from: https://github.com/ucb-bar/chiseltest/

[55] Matthew J. P. Walker and Jason H. Anderson. 2019. Generic connectivity-based CGRA mapping via integer linear programming. In *IEEE 27th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM'19)*. IEEE, 65–73.

[56] Yongchen Wang, Ying Wang, Huawei Li, Cong Shi, and Xiaowei Li. 2019. Systolic cube: A spatial 3D CNN accelerator architecture for low power video analysis. In *56th Annual Design Automation Conference*. 1–6.

[57] Dhananjaya Wijerathne, Zhaoying Li, Thilini Kaushalya Bandara, and Tulika Mitra. 2022. PANORAMA: Divide-and-conquer approach for mapping complex loop kernels on CGRA. In *59th ACM/IEEE Design Automation Conference (DAC'22)*. Association for Computing Machinery, New York, NY, 127–132. DOI: https://doi.org/10.1145/3489517.3530429

[58] Dhananjaya Wijerathne, Zhaoying Li, Anuj Pathania, Tulika Mitra, and Lothar Thiele. 2021. HiMap: Fast and scalable high-quality mapping on CGRA via hierarchical abstraction. *IEEE Trans. Comput.-aid. Des. Integ. Circ. Syst.* 41, 10 (2021), 3290–3303.

[59] Max Willsey, Vincent T. Lee, Alvin Cheung, Rastislav Bodík, and Luis Ceze. 2019. Iterative search for reconfigurable accelerator blocks with a compiler in the loop. *IEEE Trans. Comput.-aid. Des. Integ. Circ. Syst.* 38, 3 (2019), 407–418. DOI: https://doi.org/10.1109/TCAD.2018.2878194

[60] Su Zheng, Kaisen Zhang, Yaoguang Tian, Wenbo Yin, Lingli Wang, and Xuegong Zhou. 2021. FastCGRA: A modeling, evaluation, and exploration platform for large-scale coarse-grained reconfigurable arrays. In *International Conference on Field-Programmable Technology (ICFPT'21)*. IEEE, 157–166.

[61] Eckart Zitzler, Marco Laumanns, and Lothar Thiele. 2001. SPEA2: Improving the strength Pareto evolutionary algorithm. Retrieved from: https://api.semanticscholar.org/CorpusID:16584254

[62] Luca Zulberti, Matteo Monopoli, Pietro Nannipieri, and Luca Fanucci. 2022. Architectural implications for inference of graph neural networks on CGRA-based accelerators. In *17th Conference on Ph.D Research in Microelectronics and Electronics (PRIME'22)*. 373–376. DOI: https://doi.org/10.1109/PRIME55000.2022.9816810

Received 16 June 2023; revised 29 December 2023; accepted 28 March 2024